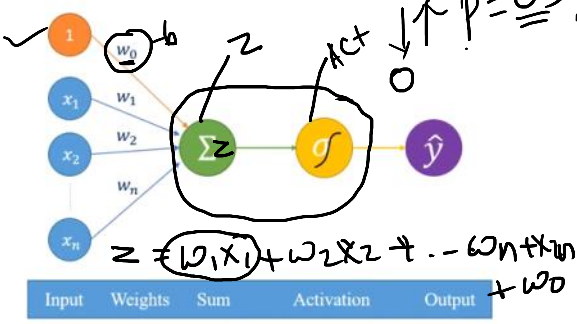
AIA

Monday, 15 December, 2025 08:59 AM

$z = wx + b$



z

Act

p = 0.5

1

0

$z = w_1x_1 + w_2x_2 + \dots w_nx_n + w_0$

$x_1 \; x_2 \; y$ → Binary 0/1   multi

R = 0.1

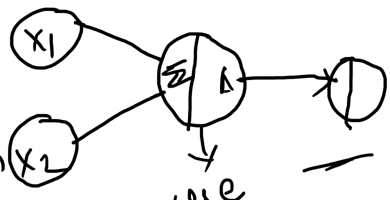
G = 0.2

B = 0.7

0.1–0.9

0–1

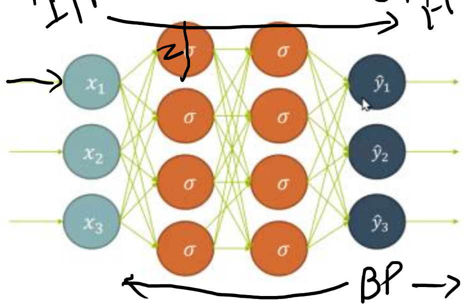
I/P   HL1   O/P



HL1, HL2 ... HL3

I/P   HL1   HL2   O/P

FP



loss ↑ 1000....

↓ 0......

1 epoch/iteration = FP + LOSS + BP

100/50

BP → optimizers

ACTIVATION FUNCTIONS   FP

Why Activation functions?
Activation functions are the mathematical functions in neural network which help in activating the neuron.

Activation functions used in Neural network are

- Linear
- Sigmoid
- Softmax
- Relu
- tanh
- Leaky relu

→ O/P

→ HL

Linear function

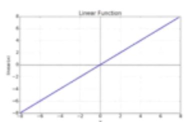
$f(z) = z$ $z = wx + b$

range= $-\infty$ to $+\infty$ $-\infty$ $+\infty$

- Used only for linear data.

10.5 L

-1.5 L



Sigmoid function → Binary

p = 0.5   1 ← → 0

Σz

$h(z) = \frac{1}{1+e^{-z}}$ $\frac{1}{1+e^{5}}$

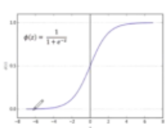
range=[0, 1]
- Used for non linear data and for binary classification.
- It is used only in output layers.

z = 5

z = -0.1

z = 0   0.5

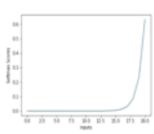
$\frac{1}{1+e^{-(0.1)}}$



Softmax function → multiclass

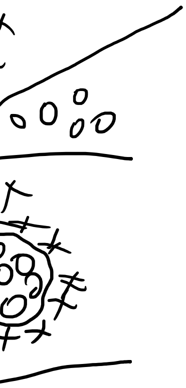
range=[0, 1]
- Used for multiclassification problems.
- It is used only in output layers.

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$



Softmax Activati

- Softmax activation function i
which of a fixed set of classe



on function

is often used in classification tasks, where the goal is to predict
s (more than two classes) a particular sample belongs to.

Relu function

$0, 2$

$f(z) = max(0, z)$

Or

$$f(z) = \begin{cases} z, & z > 0 \\ 0, & z < 0 \end{cases}$$

range=$[0, \infty]$

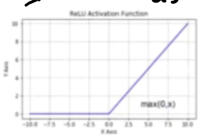
- Most used activation function
- It is used in hidden layers.

HL $\rightarrow a_1 = z_1 = 1.123$ $\rightarrow a_2 = z_2 = 0$ $a_3 = z_3 = 50$

$z_1 = 1.123$

$z_2 = -0.5$

$z_3 = 50$



Leaky Relu function

$a_1 = z_1$

$a_2 = 0.01 \times -0.5$

$= \boxed{-}$

$f(z) = max(0.01, z)$

Or

$$f(z) = \begin{cases} z, & z > 0 \\ 0.01z, & z < 0 \end{cases}$$

range=$[-\infty, \infty]$

- It is used in hidden layers.

$z_1 = 1.123$

$z_2 = -0.5$

$z_3 = 50$

Tanh function

$z = 1.123$

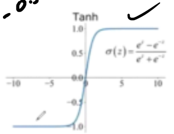
$z = -0.5$

$$f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

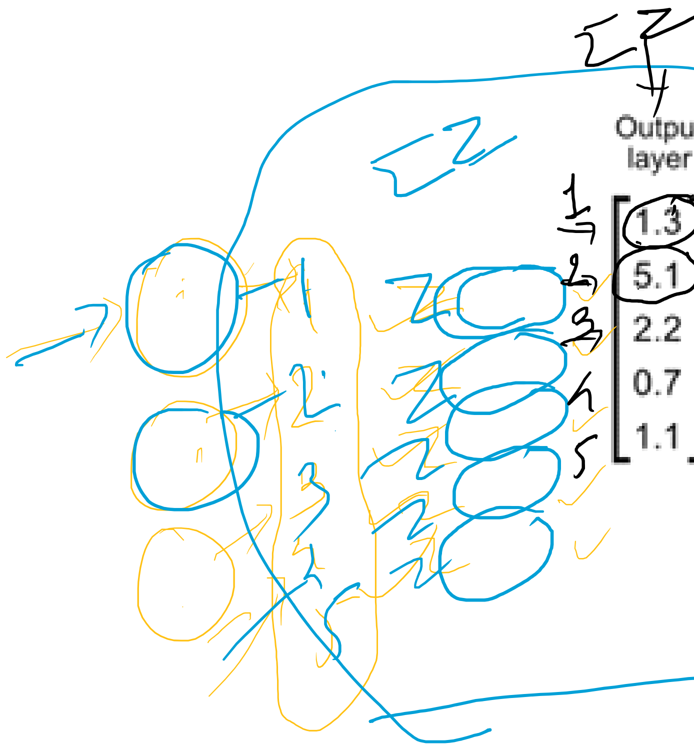
range=$[-1, 1]$

- It is used only in hidden layers.

$-1, 1$



- Softmax activation function i
which of a fixed set of classe
- The Softmax function takes i
distribution, where the sum o



X - FP

LOSS

LOSS FUNCTIONS

Why Loss function?

Loss function in neural network quantifies the difference between actual value and predicted value.

Loss functions used in Neural network are

Regression loss functions

- Mean squared error

$$MSE = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$$

- Mean absolute error

$$MAE = \frac{1}{n}\sum|y_i - \hat{y}_i|$$

Classification loss functions

- Binary cross entropy /log loss

$loss = -y_i log P_i - (1 - y_i)\log(1 - P$

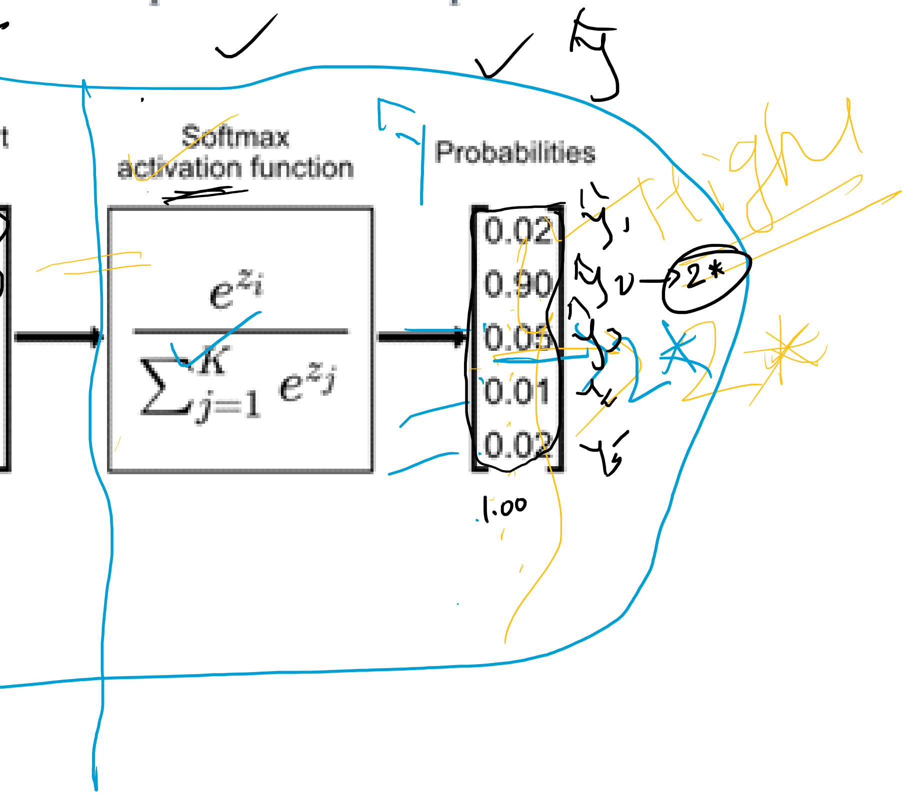
- Categorical cross entropy(softmax+

$loss = -\sum y_i \log(f(P_i))$

P_1

is often used in classification tasks, where the goal is to predict
s (more than two classes) a particular sample belongs to.

n a vector of real numbers and converts it into a probability
f all the probabilities is equal to 1.



$i)$

-CE)

$P_i <^{P_f \ 1}_{P_f \ 0}$

Optimizers

1. GD ✓
2. SGD
3. Ad
4. Ac
5. Rm

Why Optimi

Optimizers i
attributes o

Optimizers

- Gradien
- Stochast
- Adam
- Adagrad
- RMSPro

izers?

n neural network are used to minimiz

f neural network such as Weights and

used in Neural network are

t descent

tic gradient descent

l

p

e the error/ loss by modifying the
bias.

- RMSPro

GRADIENT DESCENT

How Gradient Descent works?

Gradient descent is an optimizer which is us
the error by updating the weights.

Steps:

- Randomly start with weights and works
  find optimal weight till it reaches global
  error is less.
- It uses the concept of partial derivatives
  weights till it reaches global minima i.e
  minimum

$$w_{new} = w_{old} - \alpha \frac{\partial L}{\partial w}$$

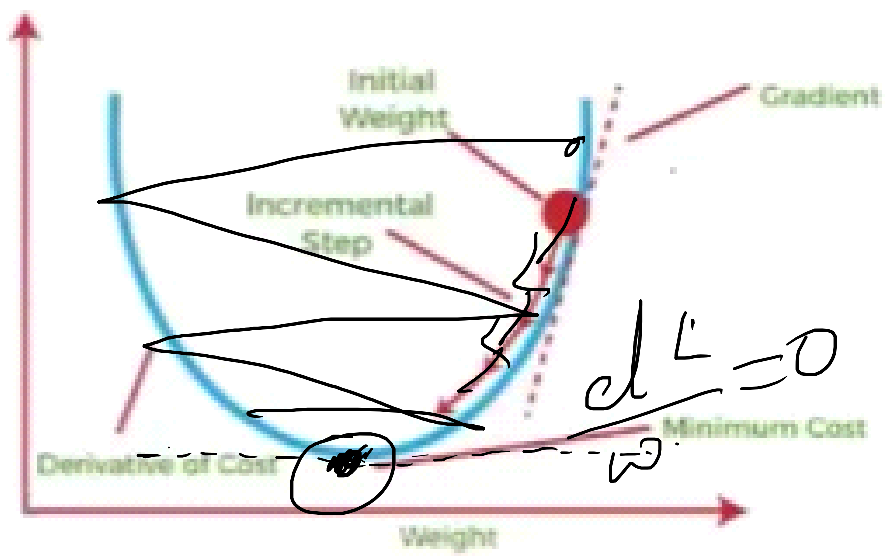
p

T

sed to minimize

iteratively to
minima where

s to update the
where loss is

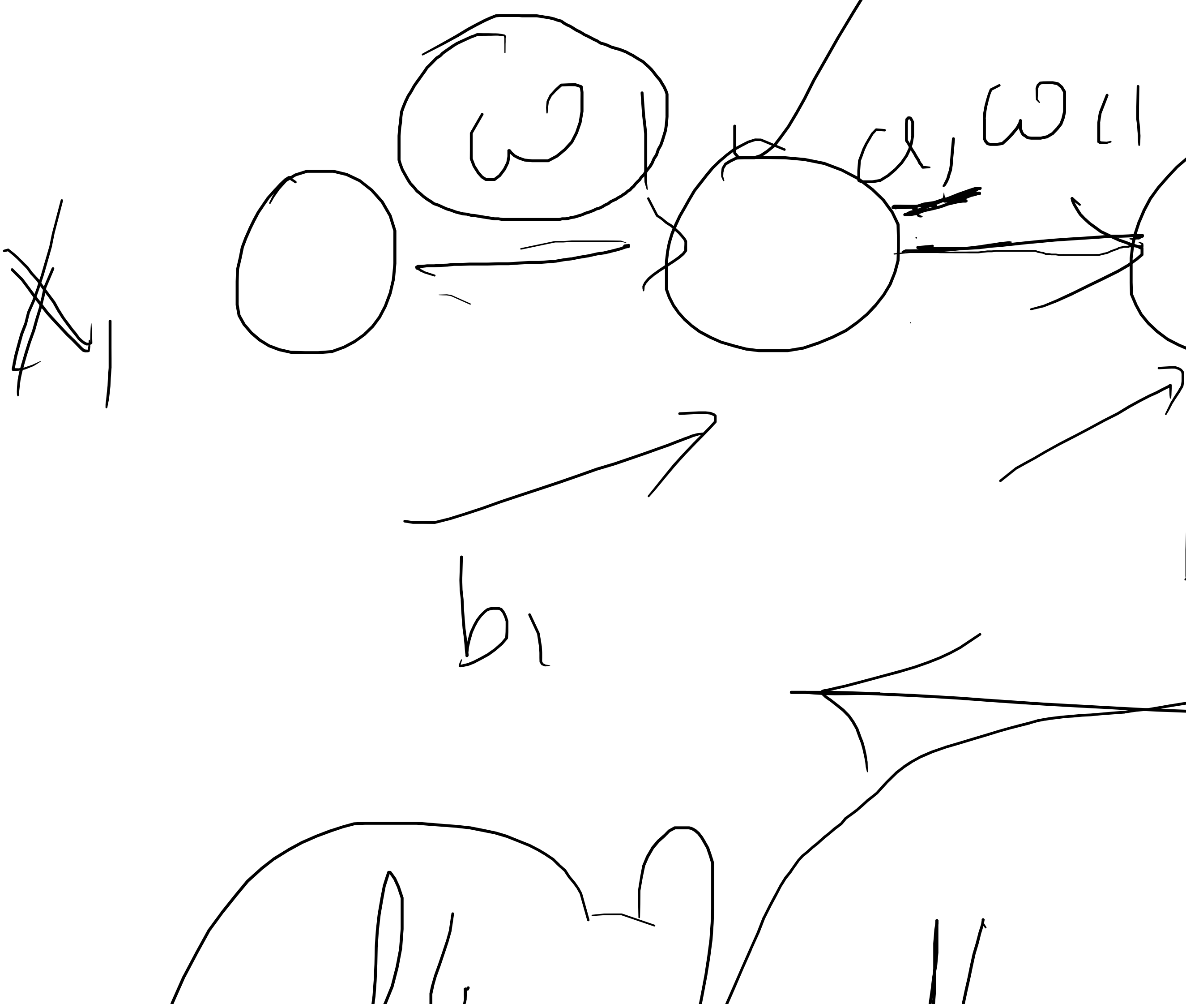


$d \quad \underline{100} = 5$

$+4$

$m \cdot 01$

I/P ✓



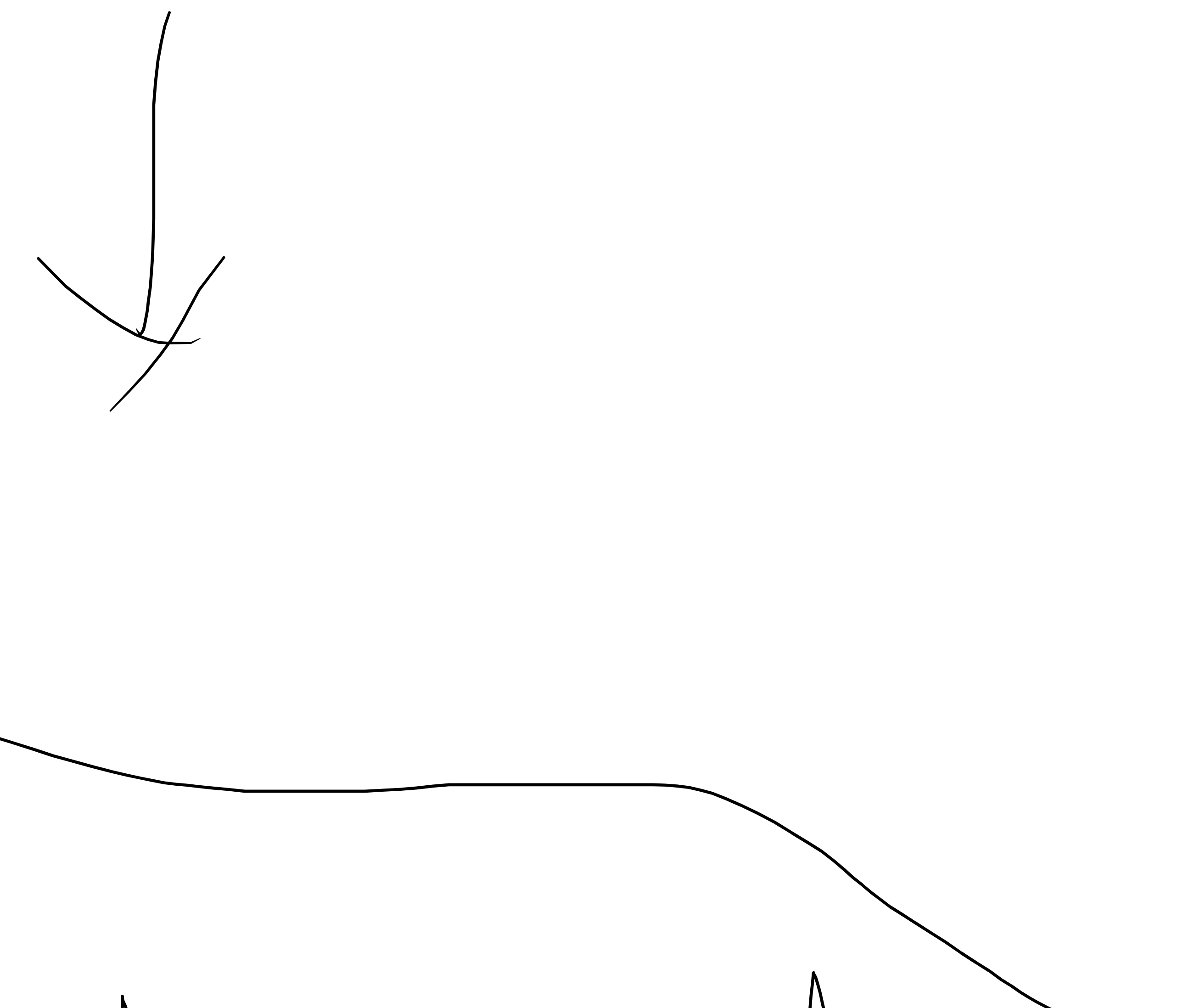
O/P

$a_i \rightarrow \hat{y} \longrightarrow$ LOSS

=

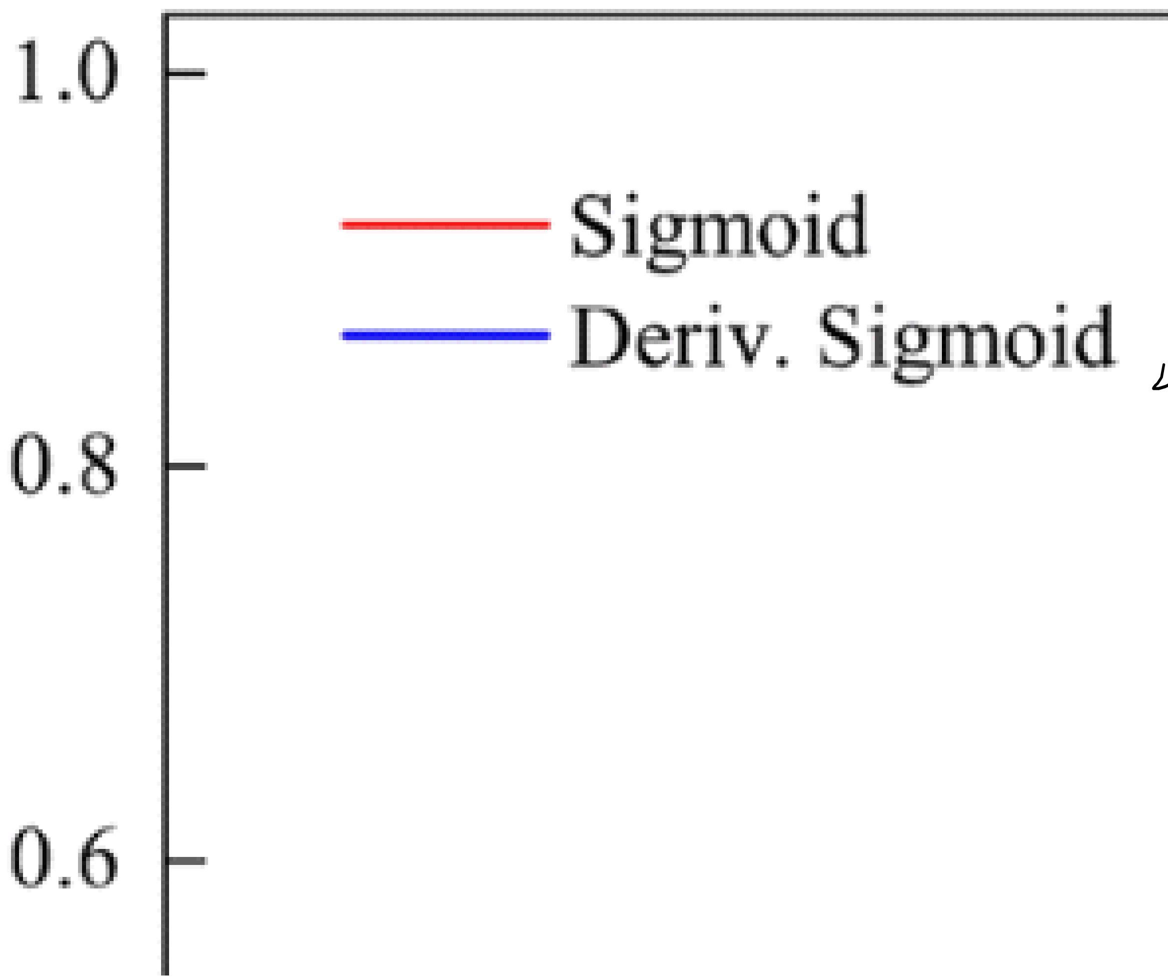
BP

$\hat{}$



$$w_{new} = w_{old} - \alpha \frac{\partial L}{\partial w}$$

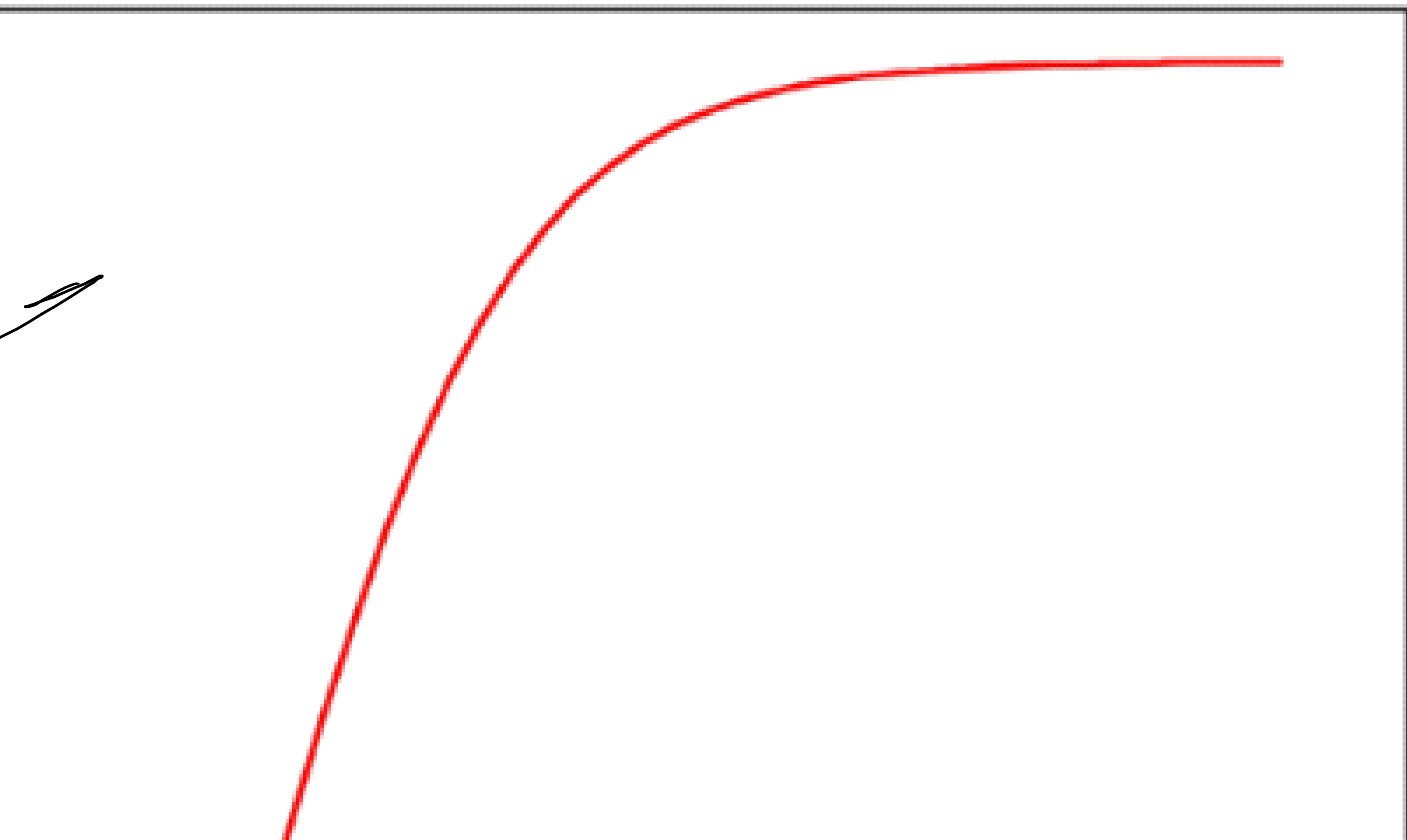
$$b_{new} = b_{old} - \alpha \frac{\partial L}{\partial b}$$



0.01

$w_n = w_{old} - (\cancel{t}\,s)$

$5 + 4 =$



$$\frac{dL}{dw_1} = \frac{dL}{d\hat{y}}$$

0.2

$w =$

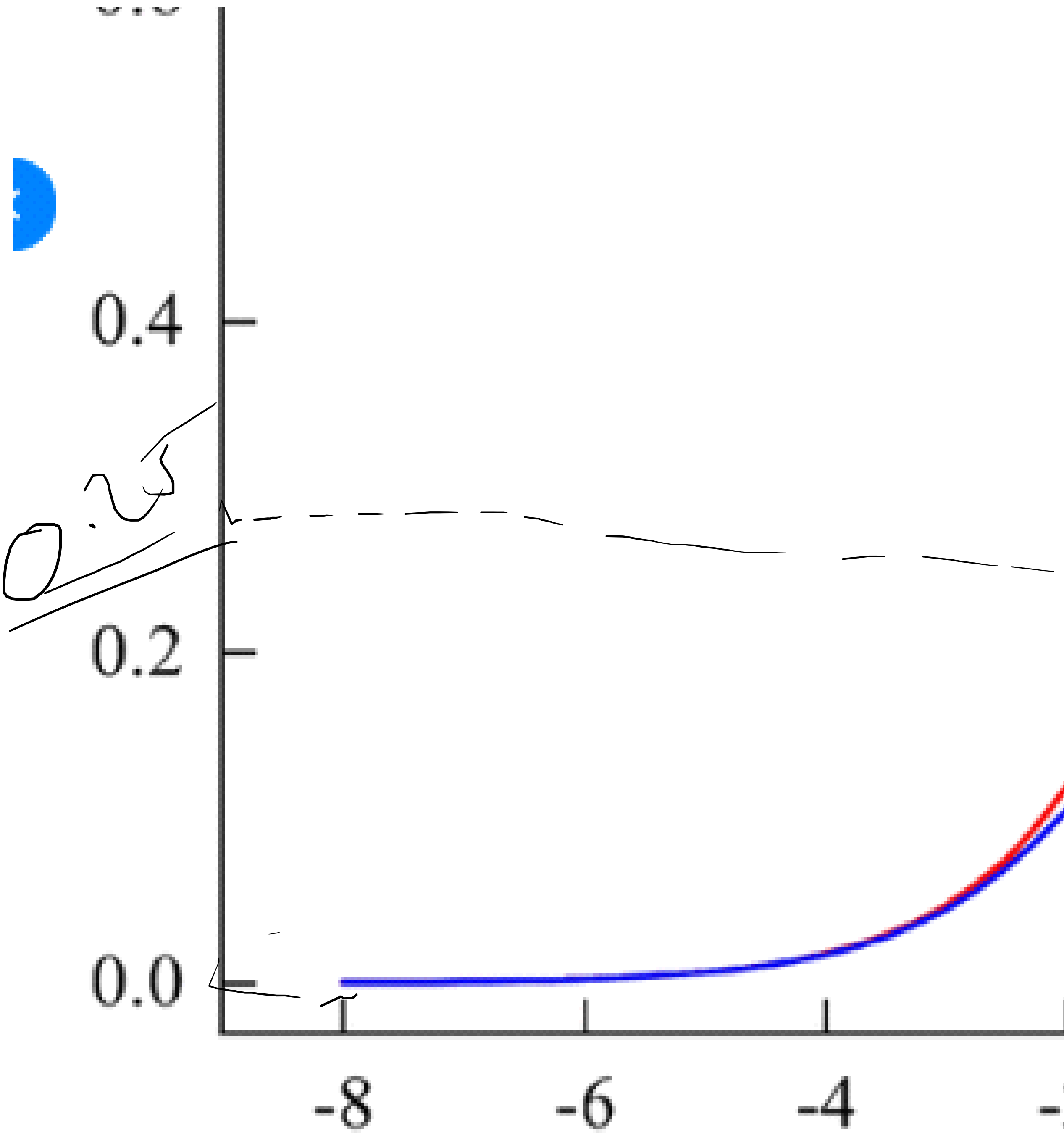
$$\times \frac{dy}{da_2} \times \frac{da_2}{dw_{11}} \times$$

$$\frac{5}{0.25}$$

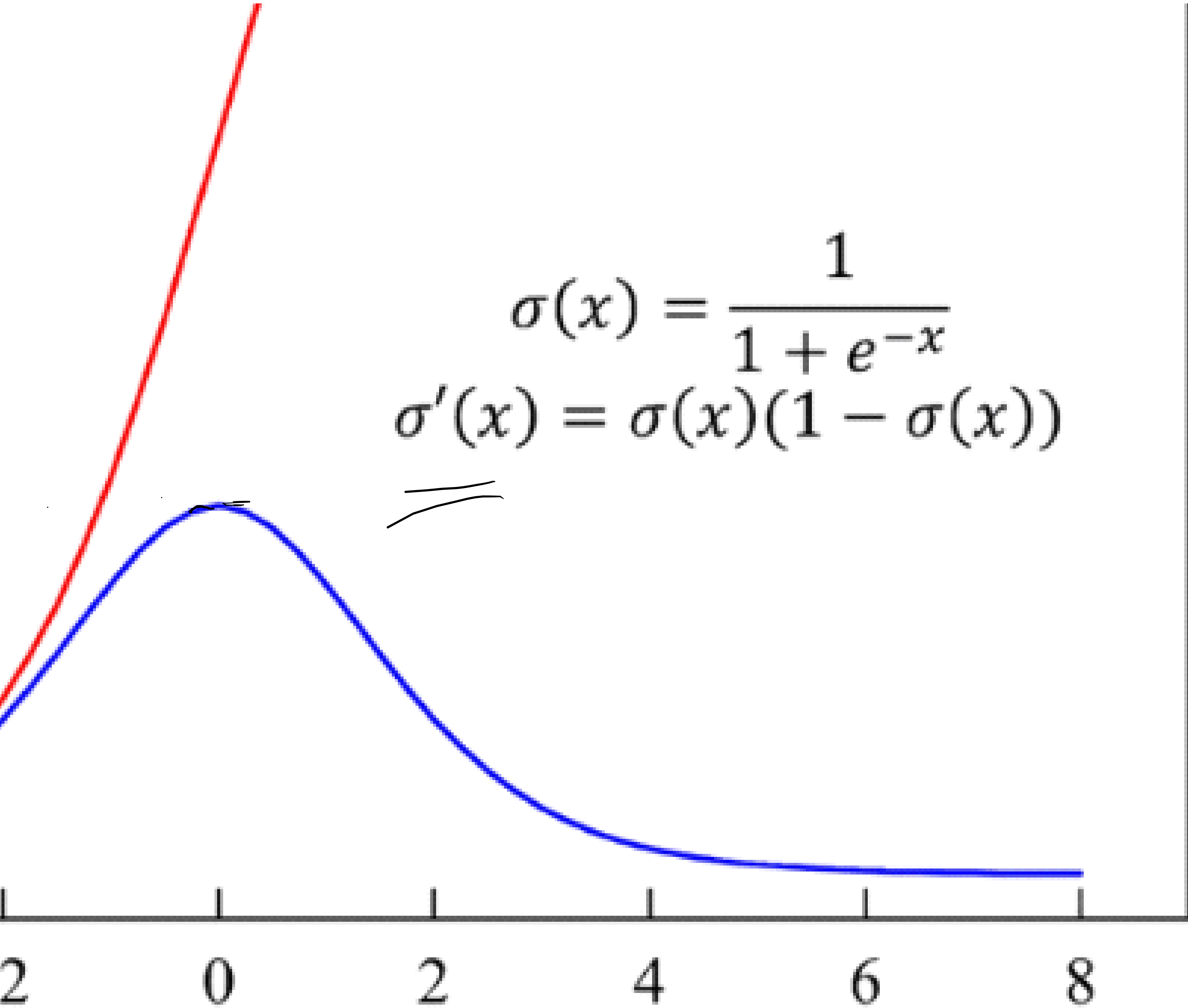
0.2

0 0

$$\frac{d\omega_{11}}{da_1} \times \frac{da_1}{d\omega_1} =$$



Cyanish



ing gradie

25×0.25

$=$

$\cong 0$

vanish

explode

ing gradi

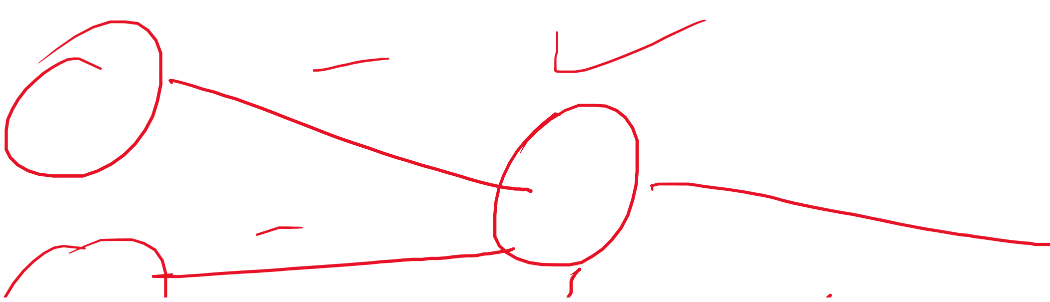
ing gradie

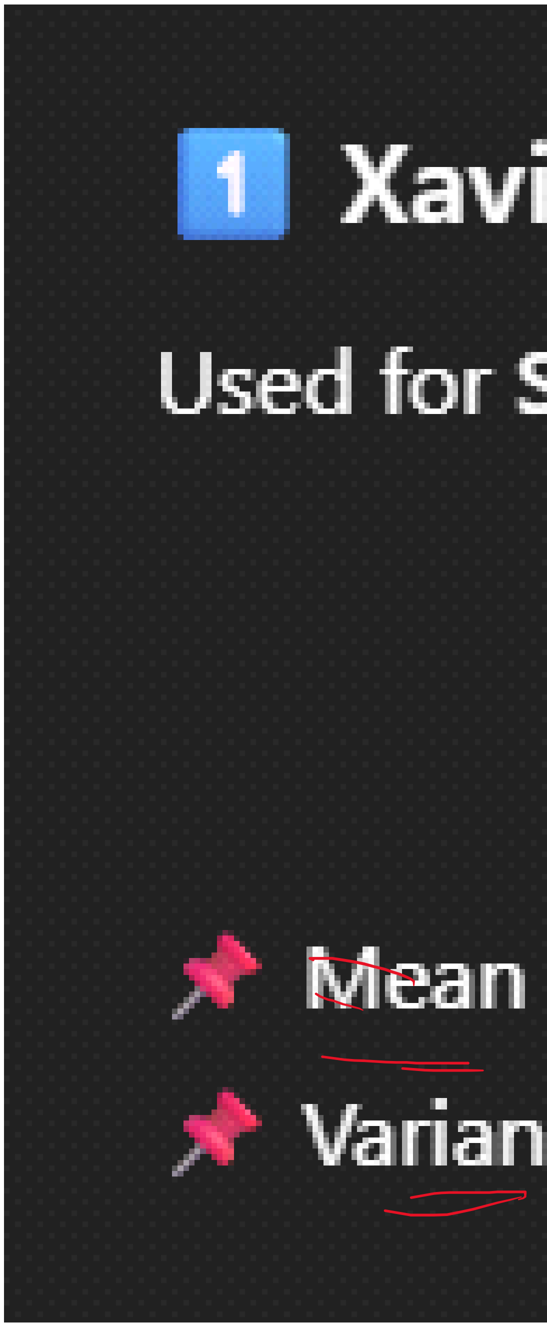
Sign

tenh

eat

oid




1 Xavi
Used for S
Mean
Varian

ier (Glorot) Normal Initialization

Sigmoid / Tanh

$W \sim N$

$= 0$

$ce = \frac{2}{n_{in}+n_{out}}$

Var(w)

$(0, 0.33)$

$$\left(0, \frac{2}{n_{in} + n_{out}}\right)$$

$$\frac{2}{4+2} = \frac{2}{6} = 0.33$$

- Number of inputs

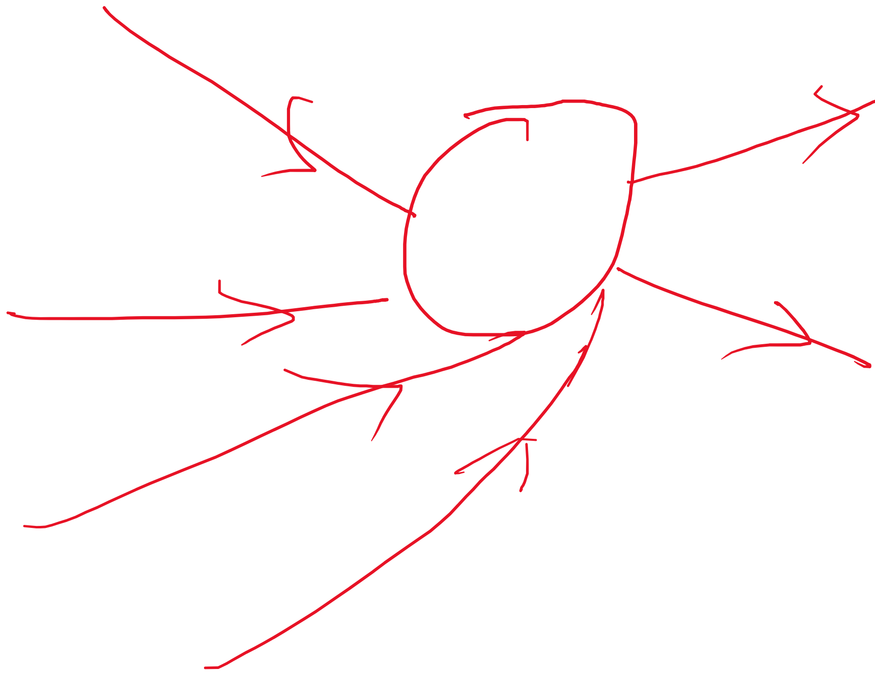
$$n_{in} = 4$$

- Number of outpu

$$n_{out} = 2$$

s:

ts:




2
Used

Xavier (Glorot) Uniform Initializ

for **Sigmoid / Tanh**

$$w \sim \mathcal{U}\left(-\right.$$

zation

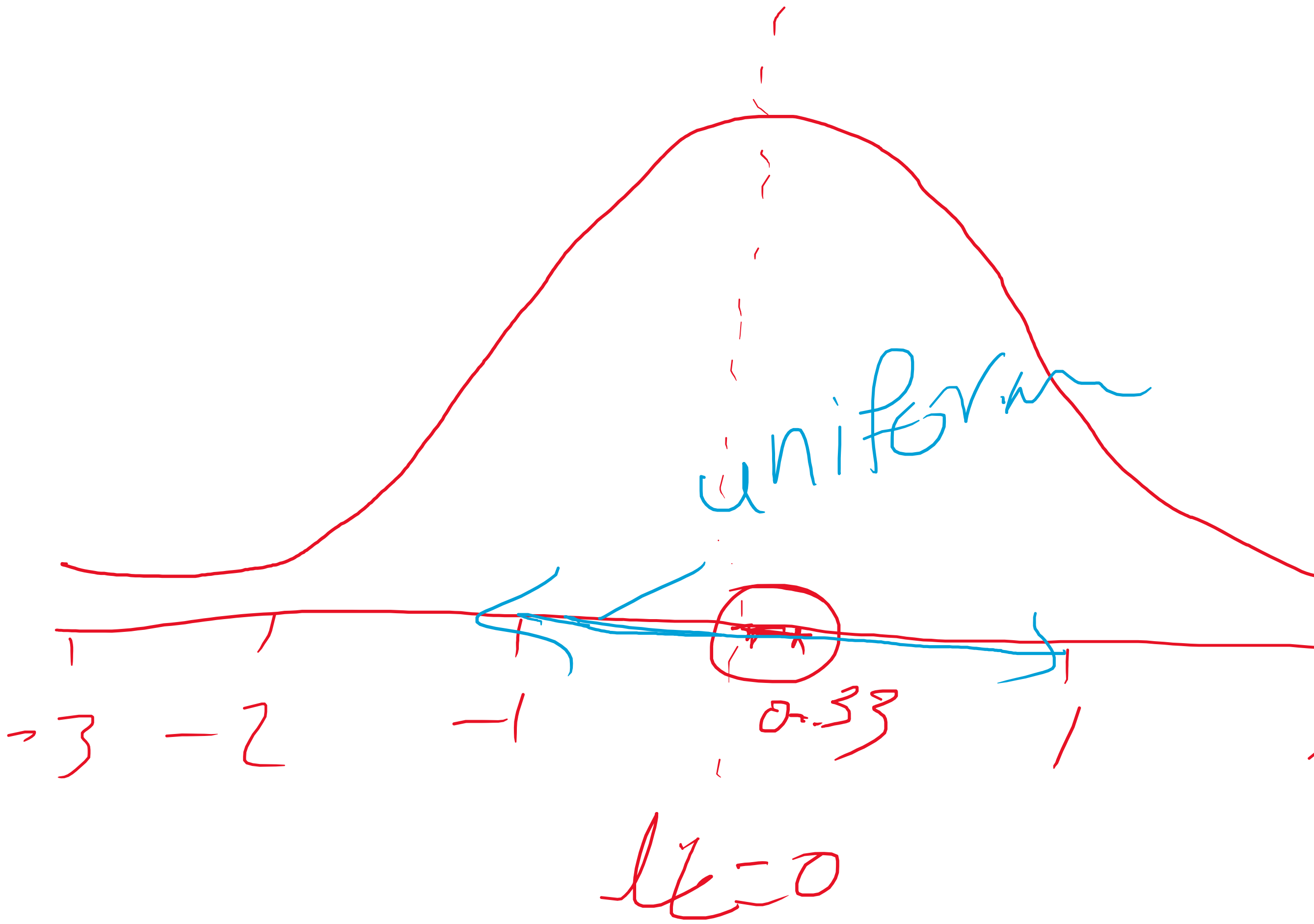
-1

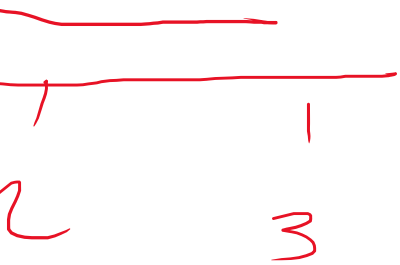
$(-1, 1)$

$-\sqrt{\frac{6}{6}}$

1

$$\sqrt{\frac{6}{\underbrace{n_{in}}_{4} + \underbrace{n_{out}}_{2}}}, \sqrt{\frac{6}{n_{in} + n_{out}}})$$


uniform
-3
-2
-1
0.38
1
μ=0


2
3

He Normal Initialization

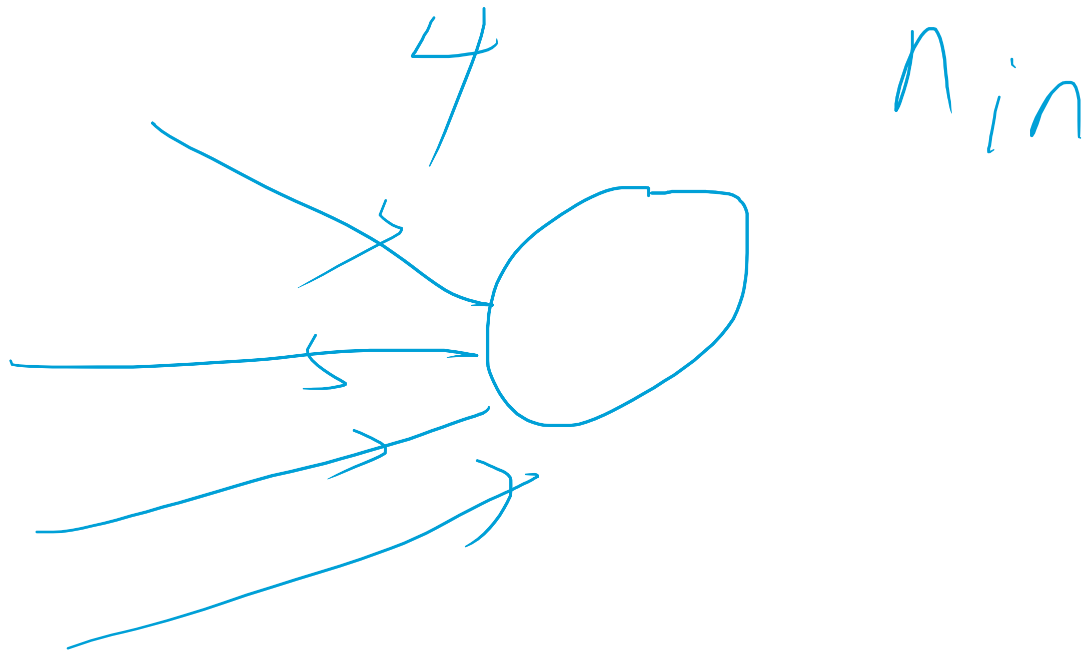
for ReLU / Leaky ReLU

Mean = 0

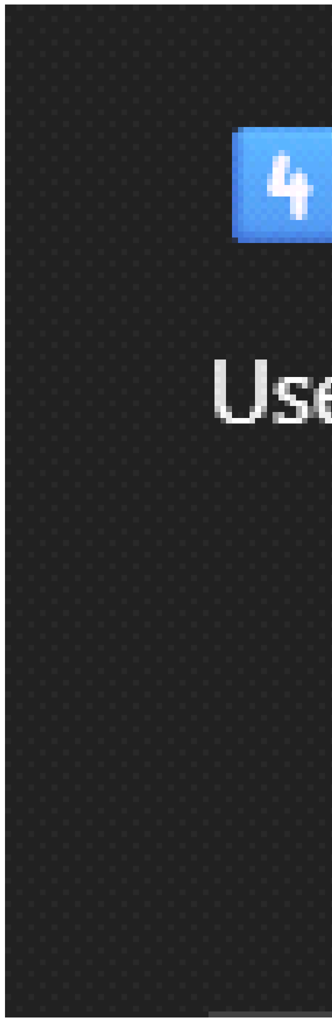
Variance

$$W \sim \mathcal{N}\left(0, \underline{\frac{2}{n_{in}}}\right)$$

$$\frac{2}{4} = 0.$$


4
n in

$= 4$


4

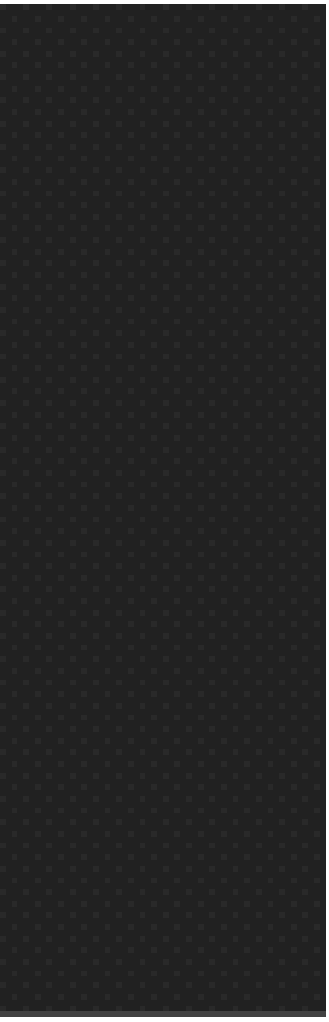
He Uniform Initialization

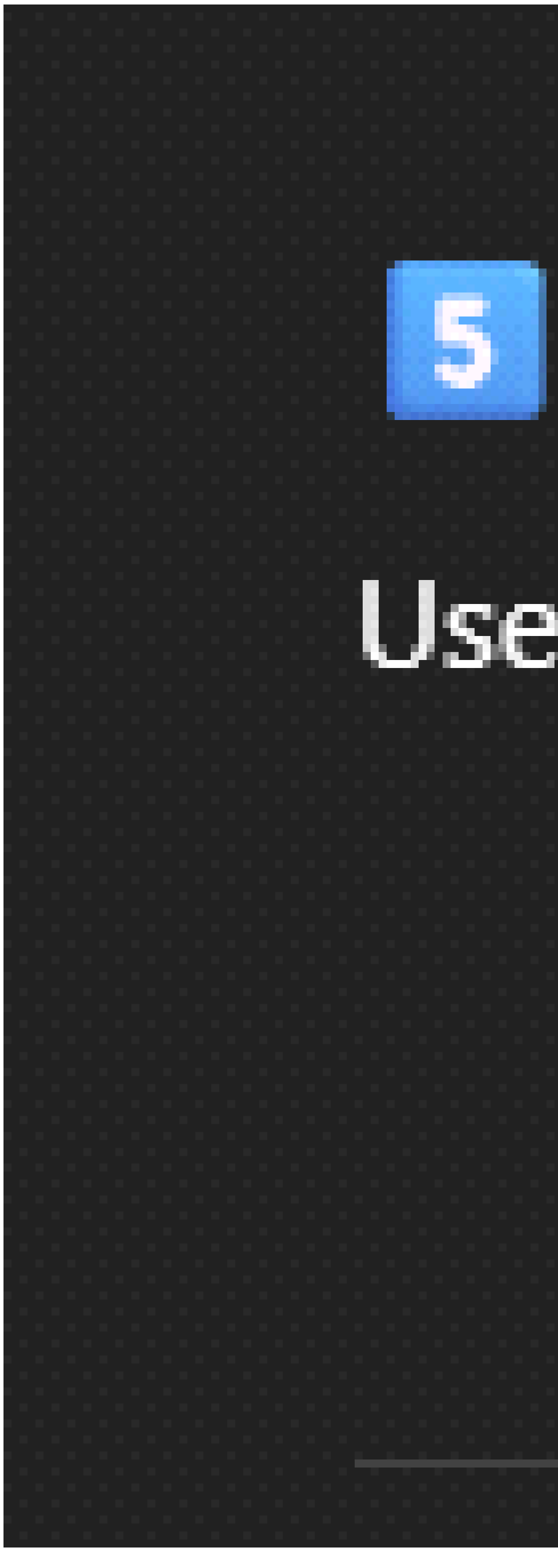
ed for **ReLU / Leaky ReLU**

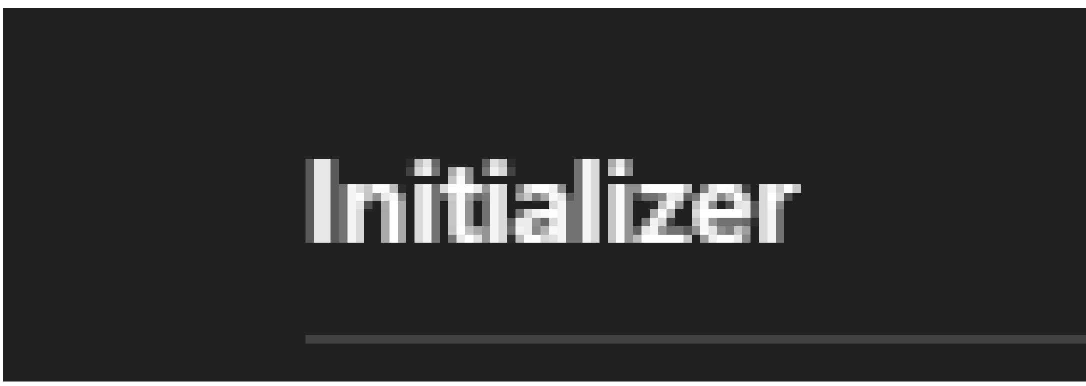
W

$$-\sqrt{\frac{6}{4}}, \quad \sqrt{\frac{6}{4}}$$

$$\sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{in}}}, \sqrt{\frac{6}{n_{in}}}\right)$$




5
Use


Initializer

LeCun Normal Initiali

ed for SELU

Distribution

zation (for SELU)

$$W \sim \mathcal{N}\left(0, \frac{}{n}\right.$$

Formula

mean = 0

$$\frac{1}{n_{in}} = \frac{1}{4}$$

$$\frac{1}{n_{in}}$$

Best Acti


ivation

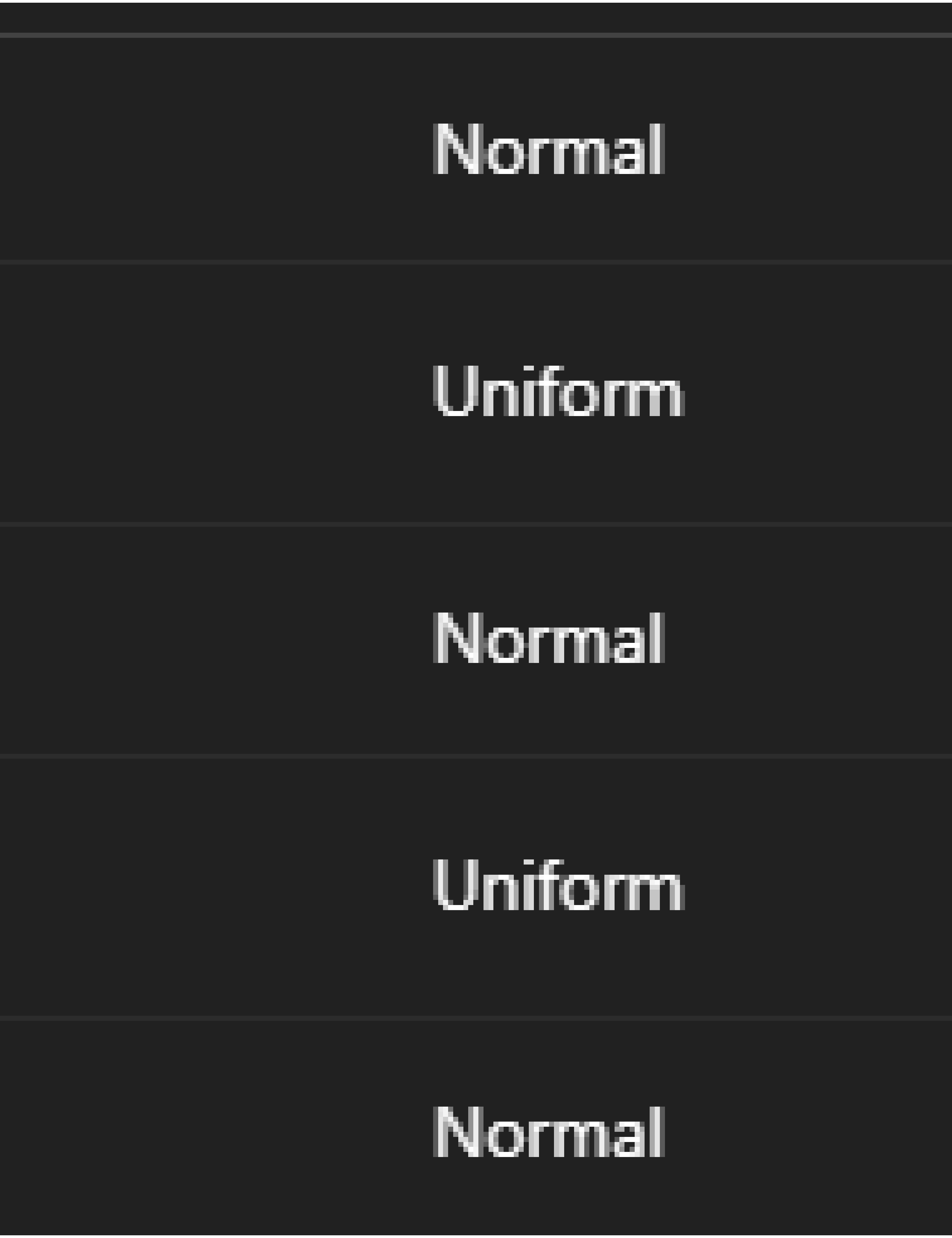
Xavier Normal

Xavier Uniform

He Normal

He Uniform

LeCun Normal


Normal
Uniform
Normal
Uniform
Normal

$$\frac{2}{n_{in}+n_{out}}$$

$$\sqrt{\frac{6}{n_{in}+n_{out}}}$$

$$\frac{2}{n_{in}}$$

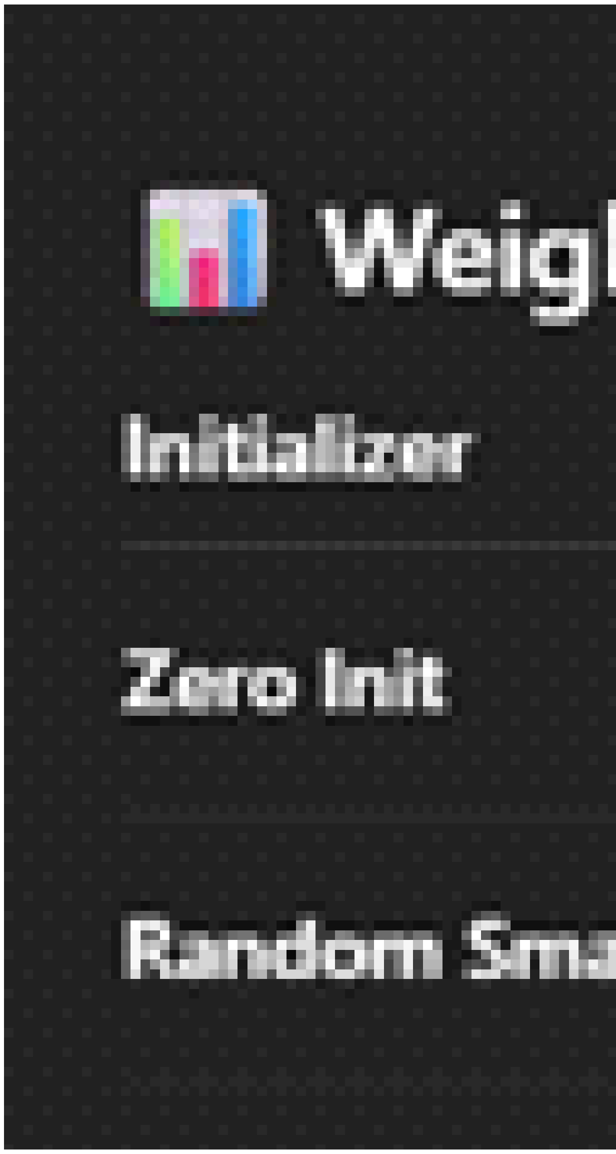
$$\sqrt{\frac{6}{n_{in}}}$$

$$\frac{1}{n_{in}}$$

Sigmoid,
Sigmoid,
ReLU
ReLU
SELU

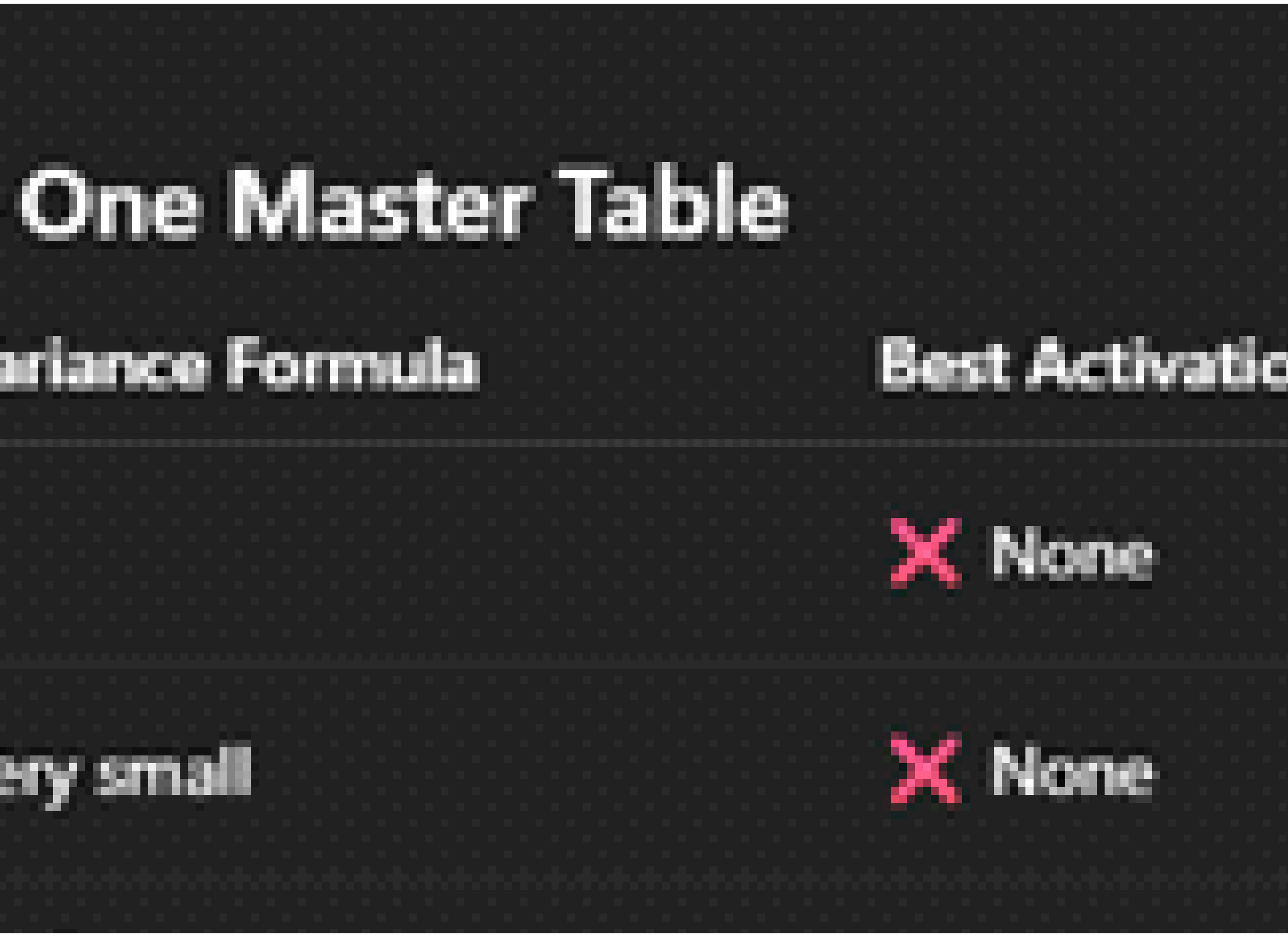
Tanh

Tanh

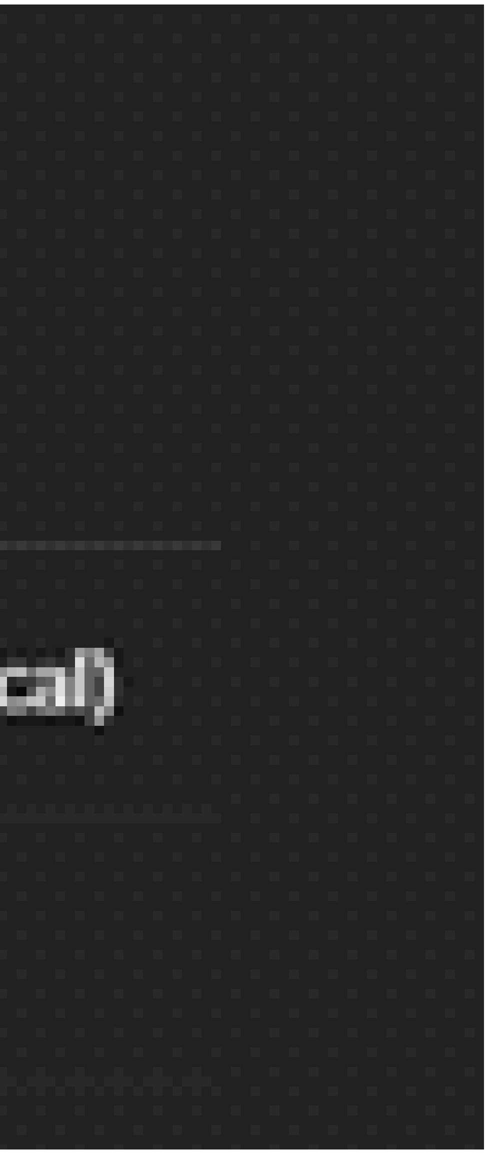

Initializer
Zero Init
Random

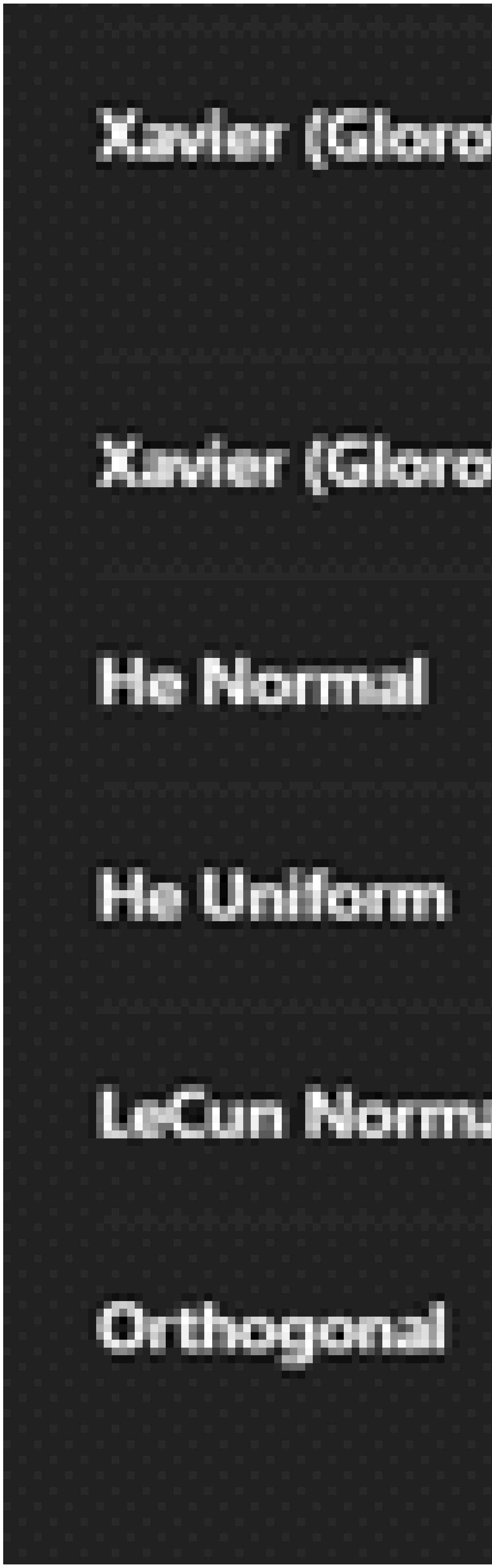
ht Initialization Techniques —

	Weight Distribution	V
	All weights = 0	0
ll	Normal / Uniform	V


One Master Table
ariance Formula
Best Activatio
None
ery small
None

on	Why It Works
	Breaks learning (all neurons identi
	Causes vanishing gradients

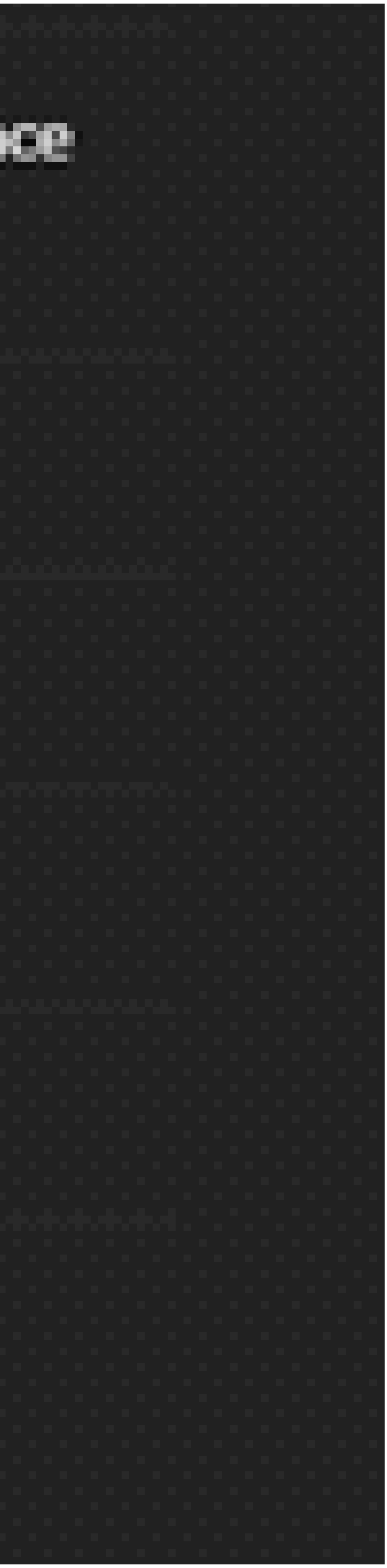

cal)


Xavier (Gloro
Xavier (Gloro
He Normal
He Uniform
LeCun Norma
Orthogonal

t) Normal	Normal (mean = 0)	
t) Uniform	Uniform	
	Normal (mean = 0)	
	Uniform	
l	Normal (mean = 0)	
	Orthogonal matrix	G

$\frac{2}{n_{in} + n_{out}}$	Sigmoid, Tanh
$\frac{6}{n_{in} + n_{out}}$	Sigmoid, Tanh
	ReLU, Leaky R
	ReLU family
	SELU
ain-scaled	RNNs, deep n

n	Keeps activation & gradient varian stable
n	Same goal, different distribution
eLU	Prevents dying ReLU
	Better gradient flow
	Preserves self-normalization
ets	Preserves norm across layers



tensorflow

load

tensor flo

1 Number

10 →

$\overline{\overline{10}}$ → 0

2 vector

encoding

scalar

dim

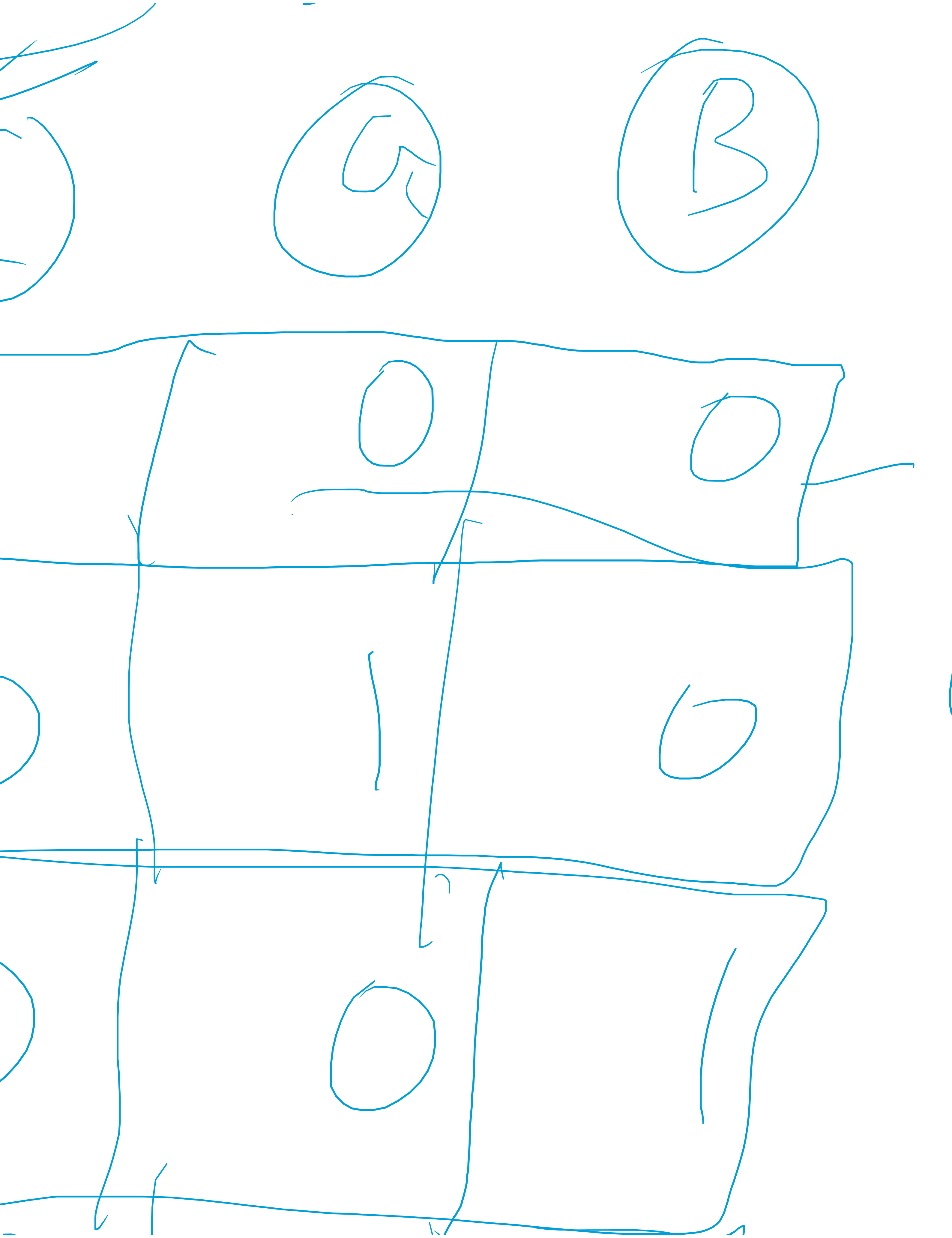
→ 1D array

y

3 matrix

2D array

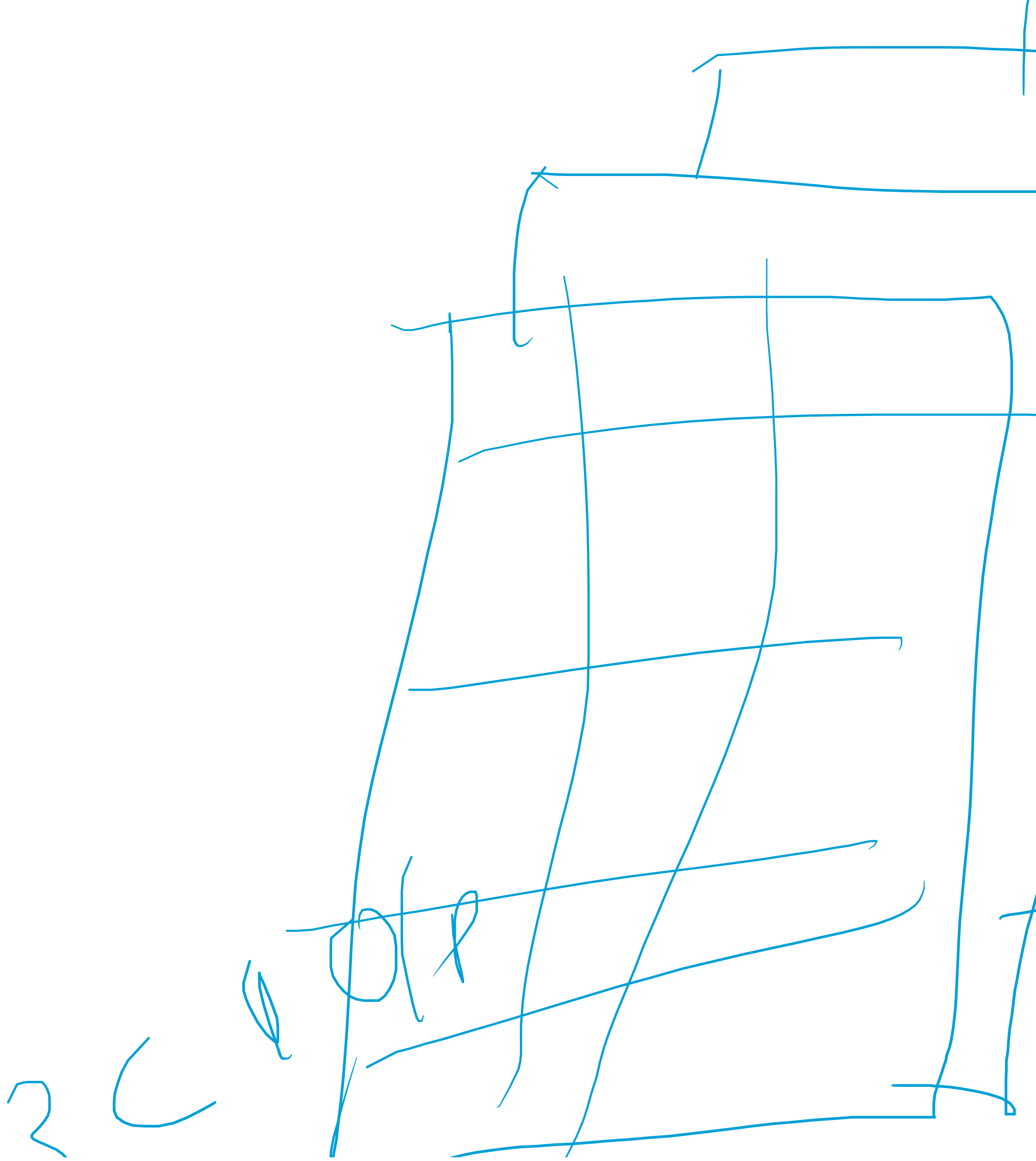
one



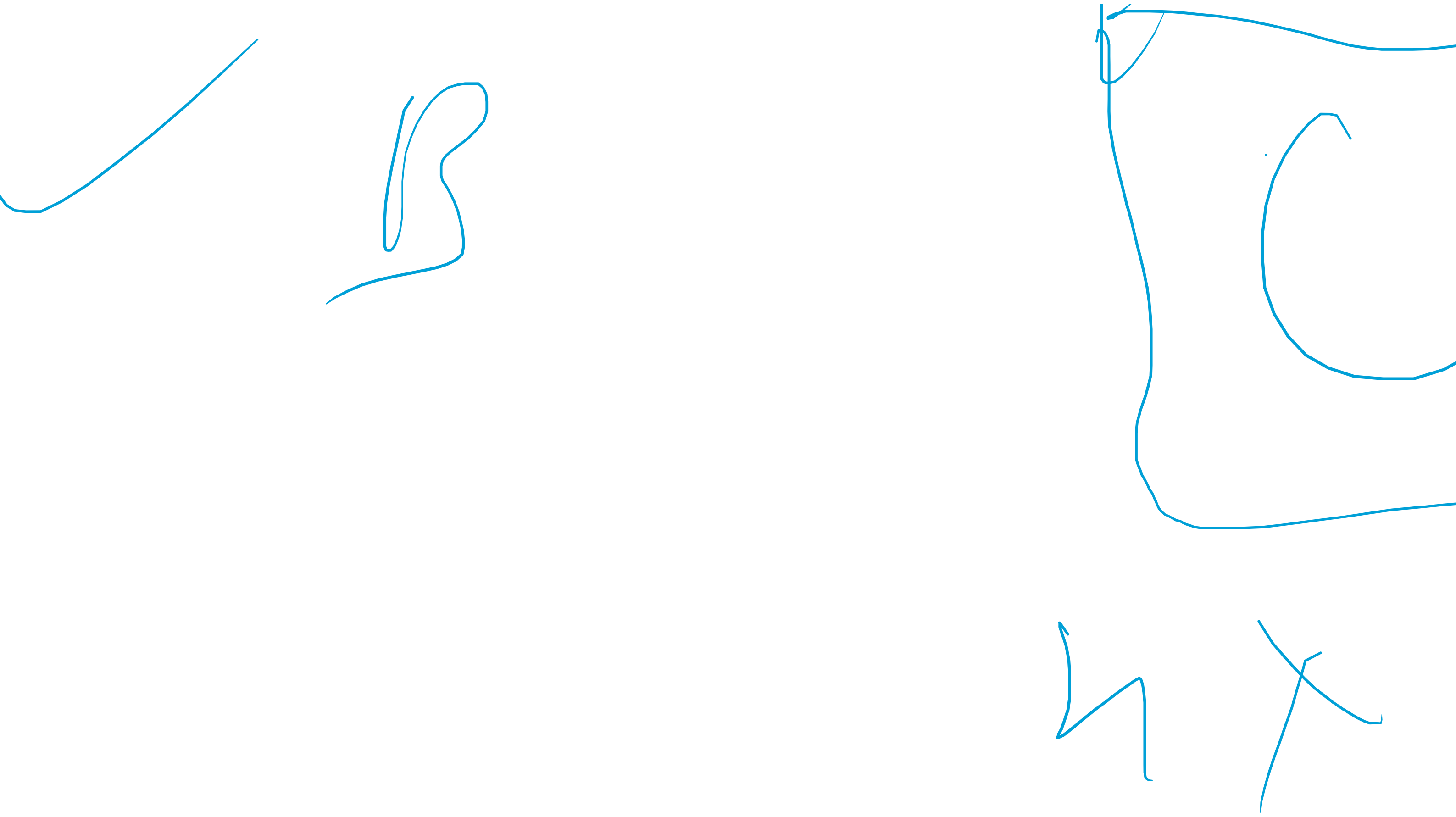
1D array

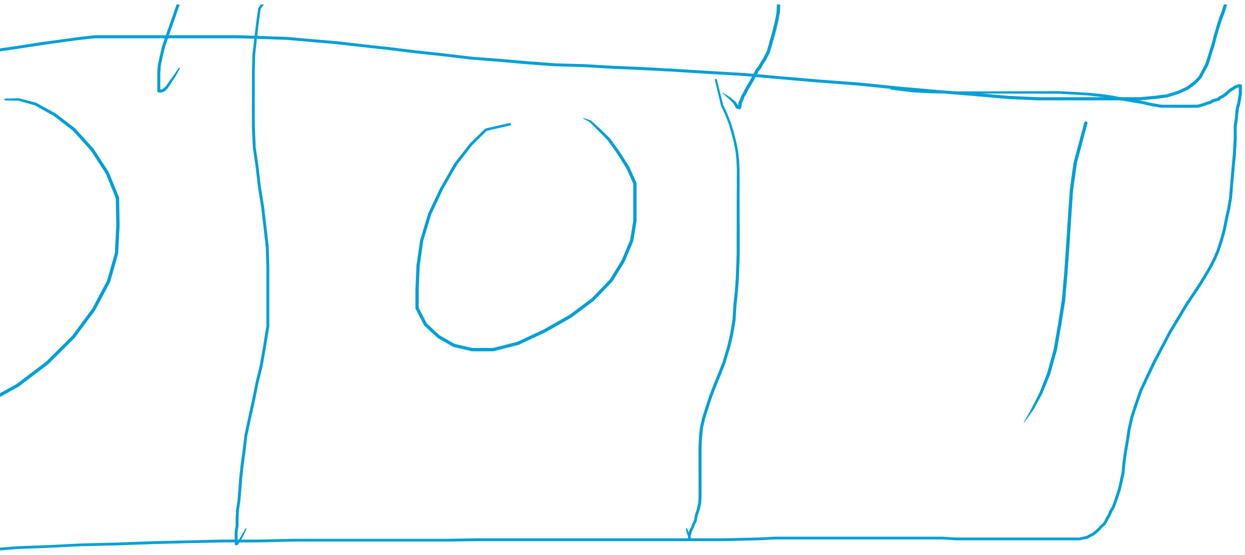
vector

4 tensor

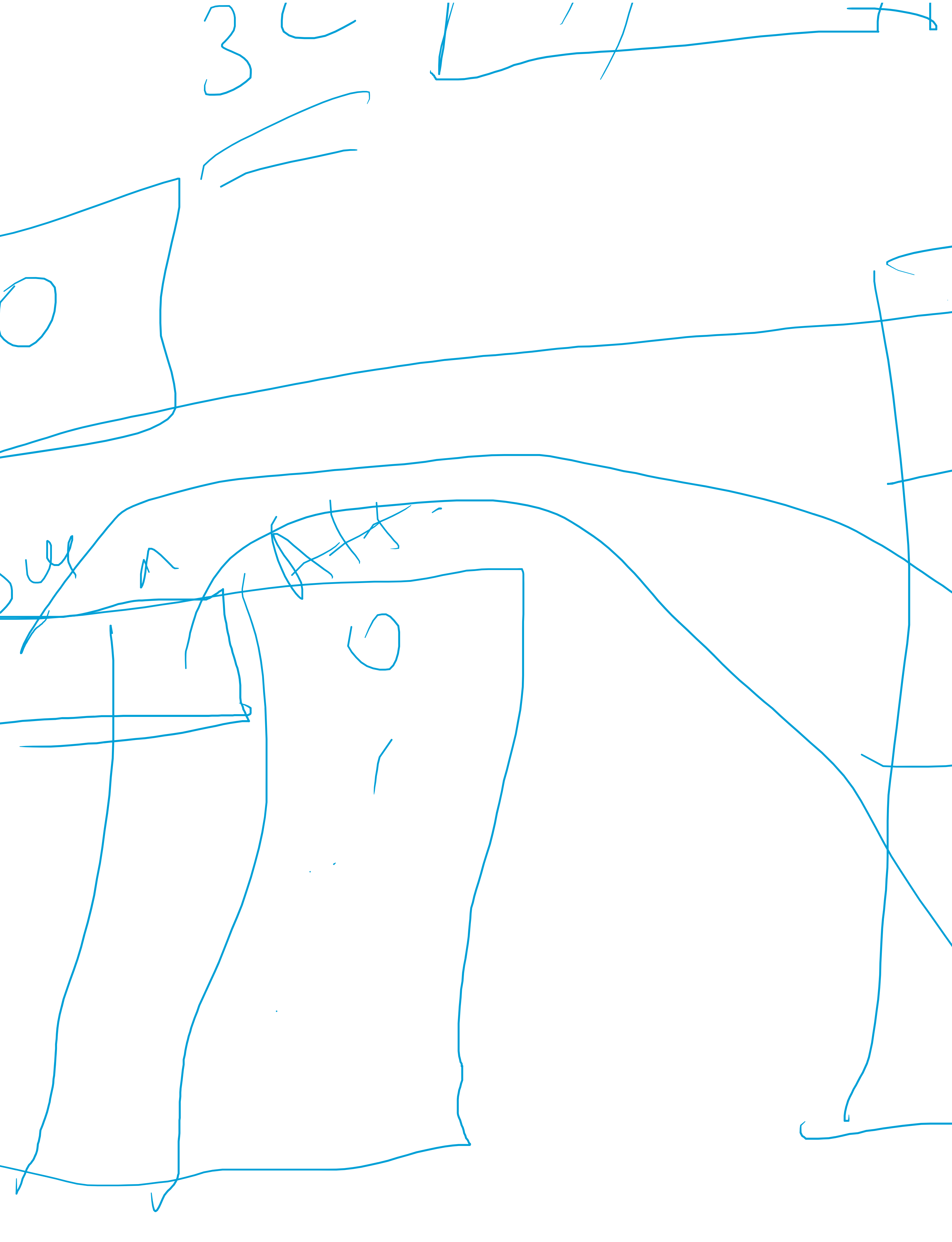


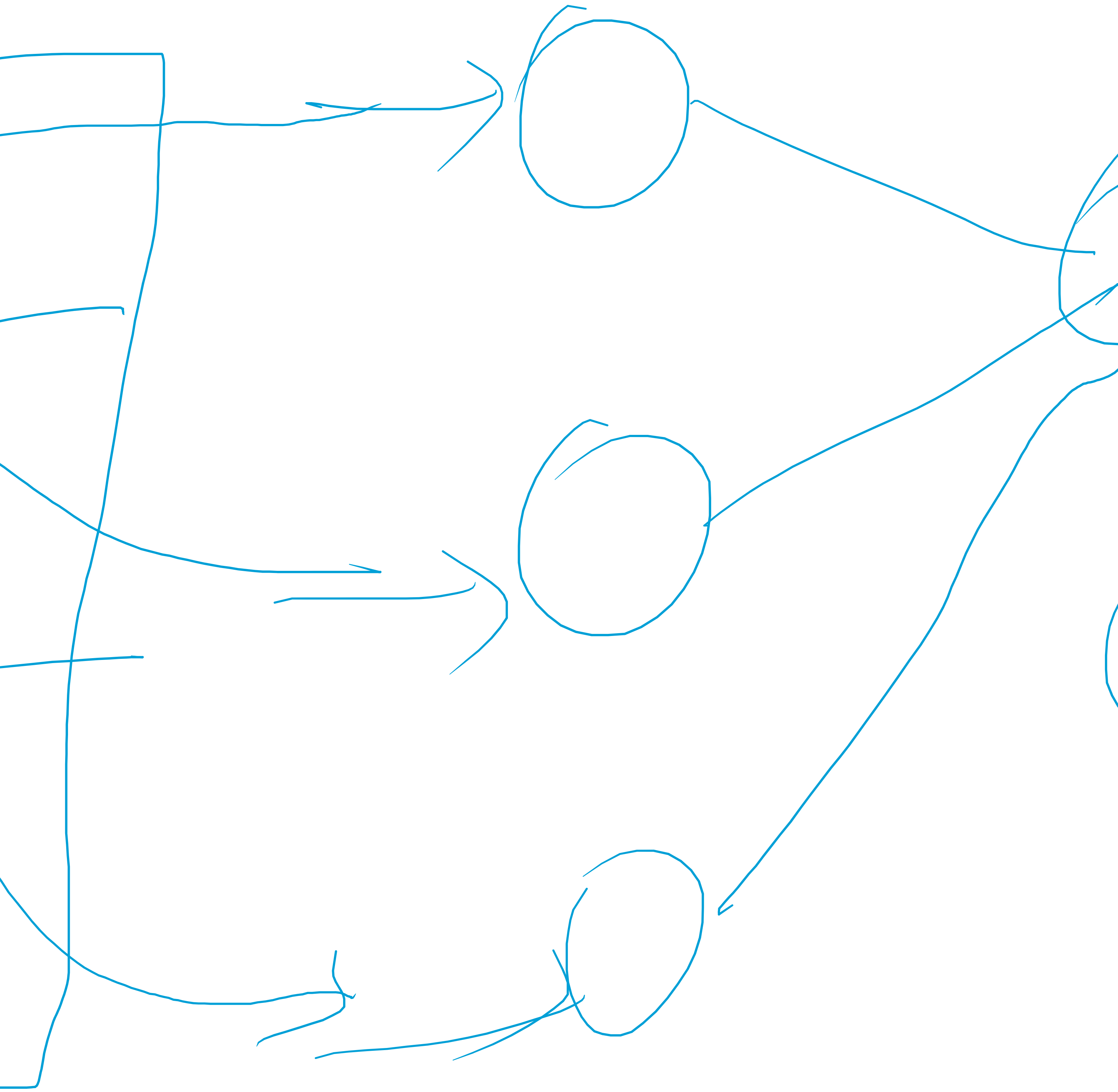
nD array

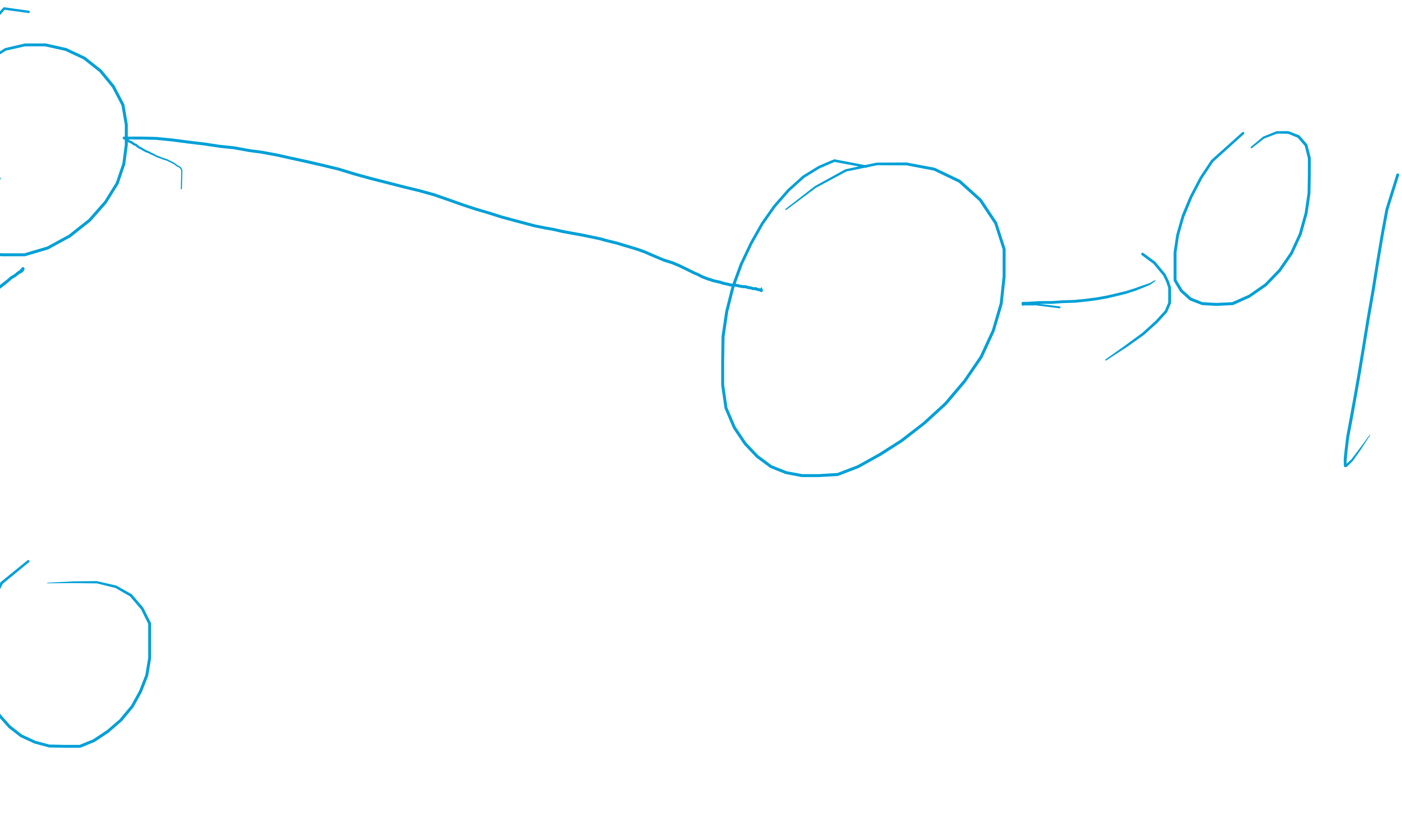


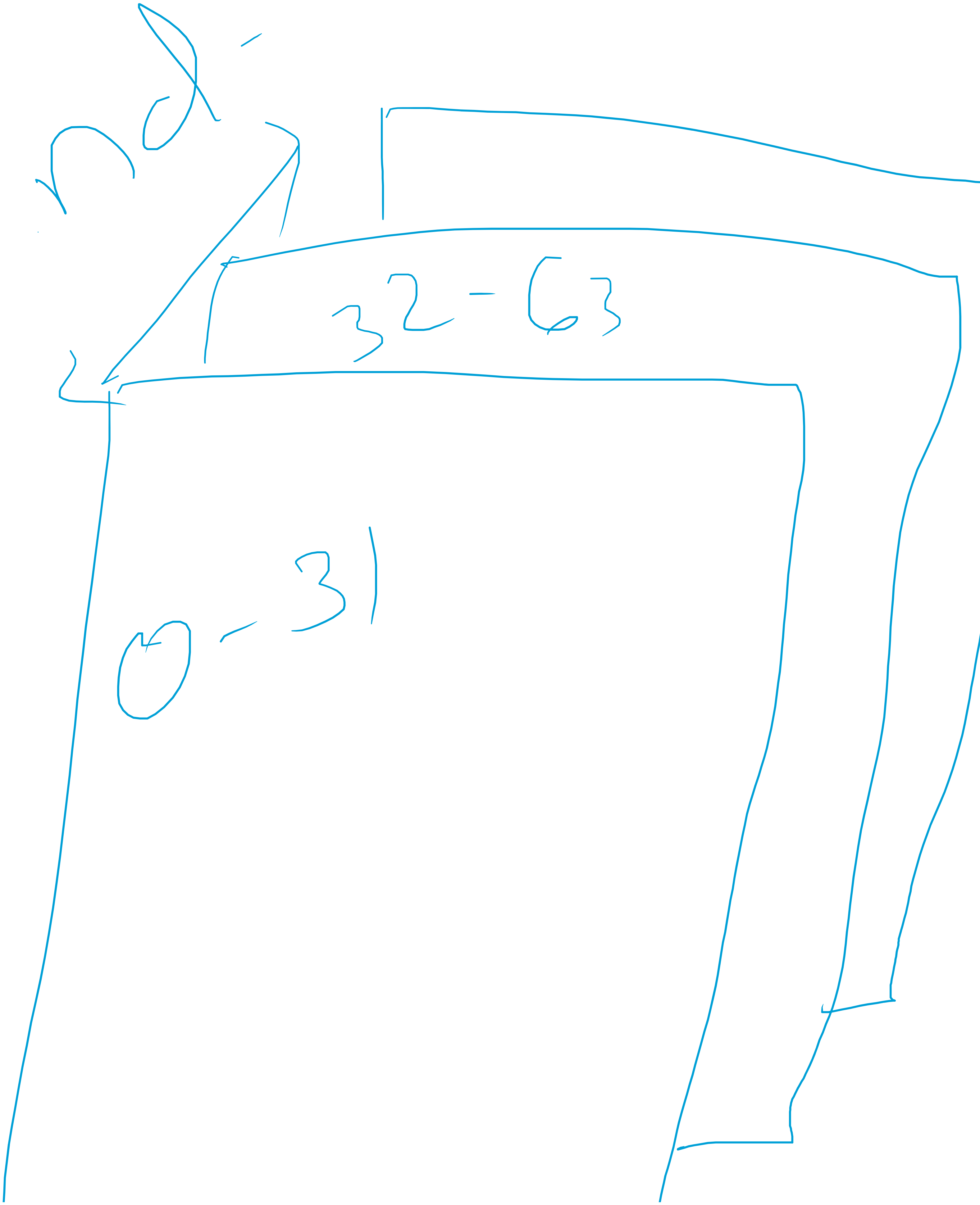


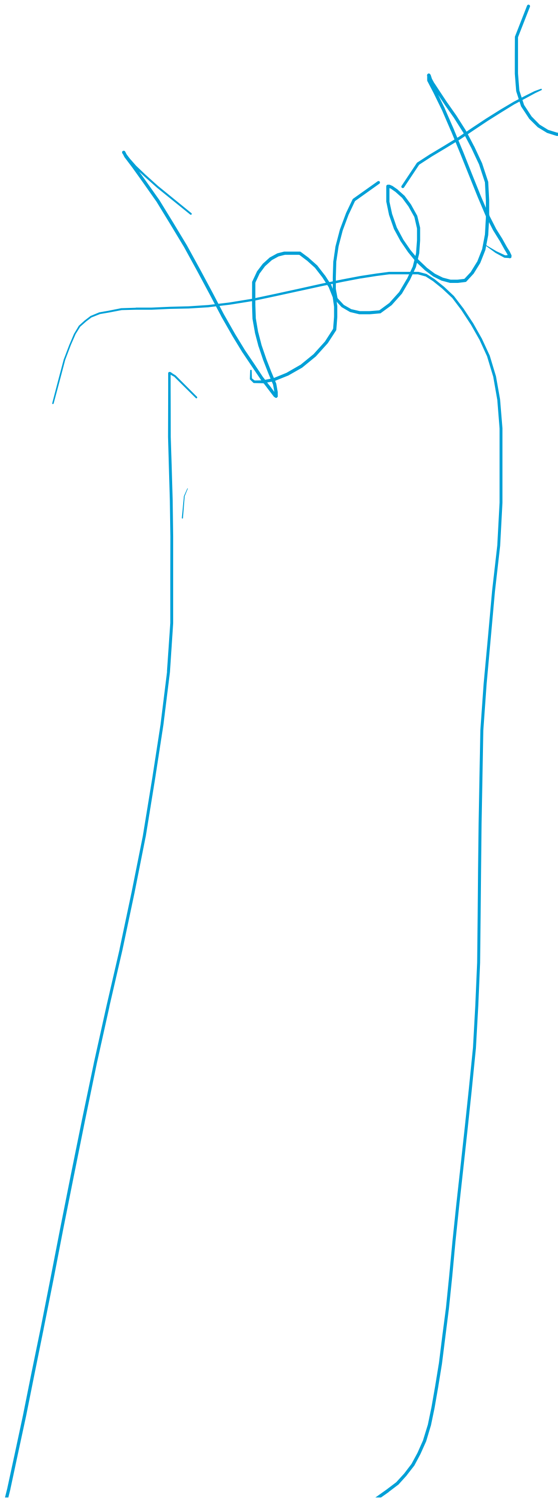
3

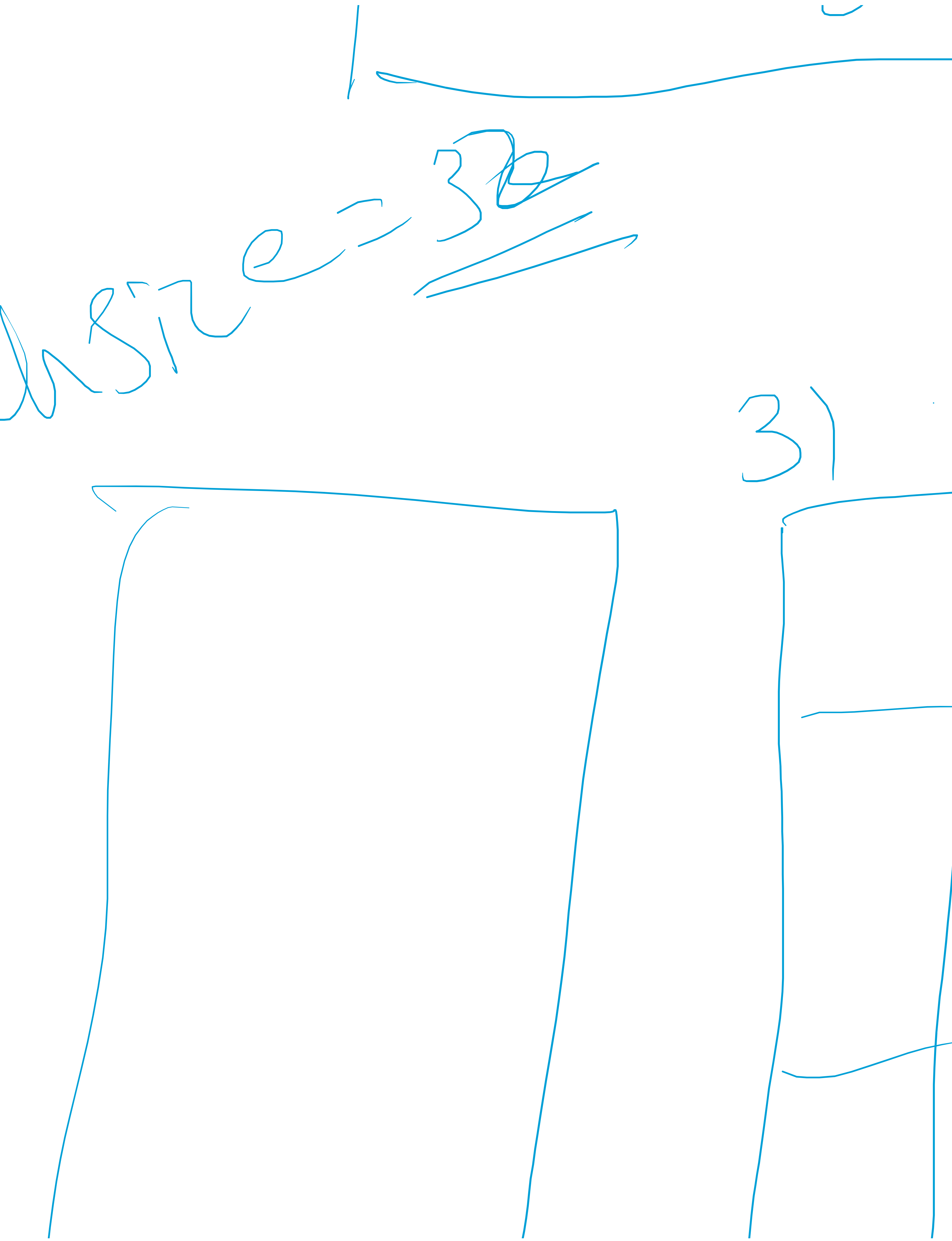


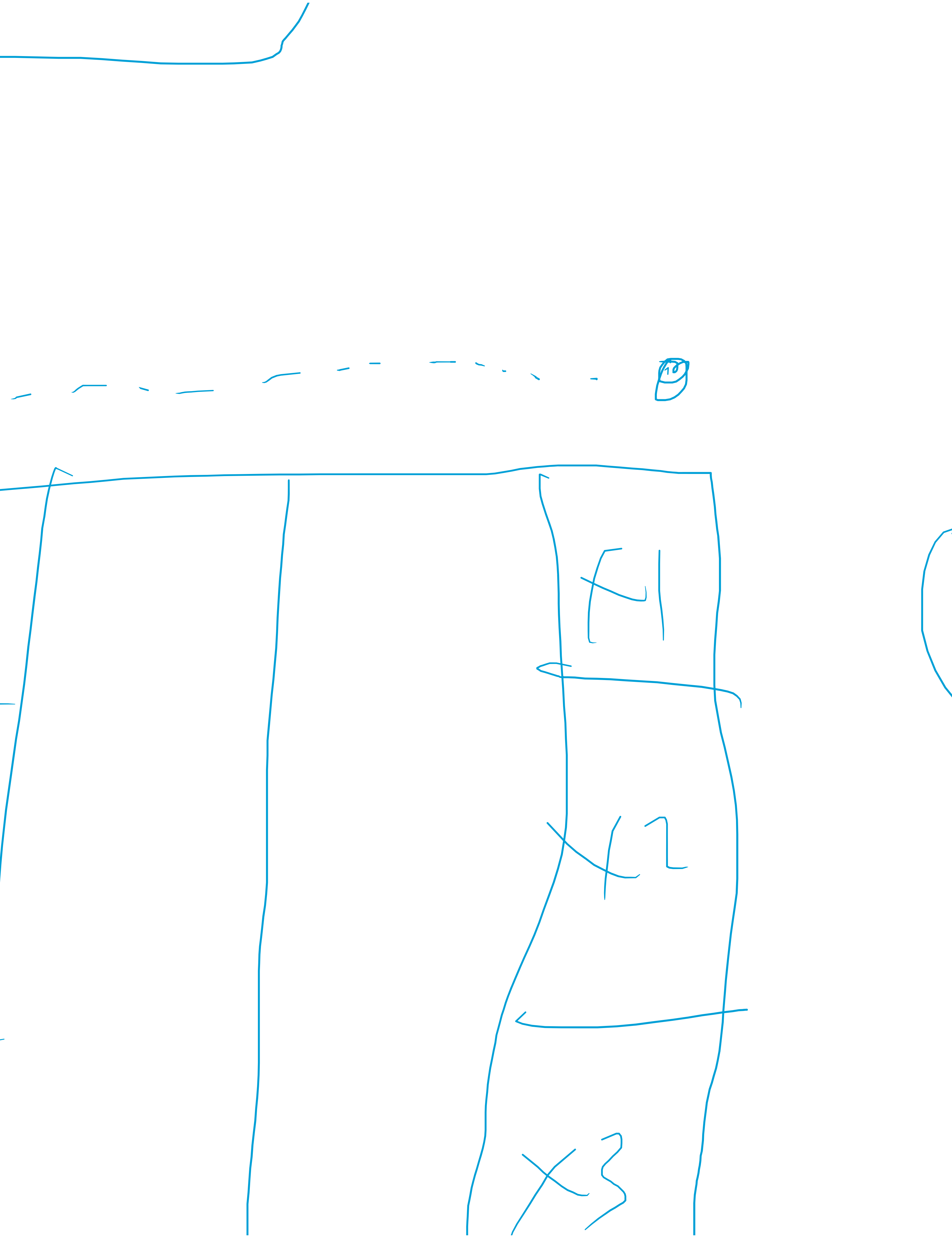


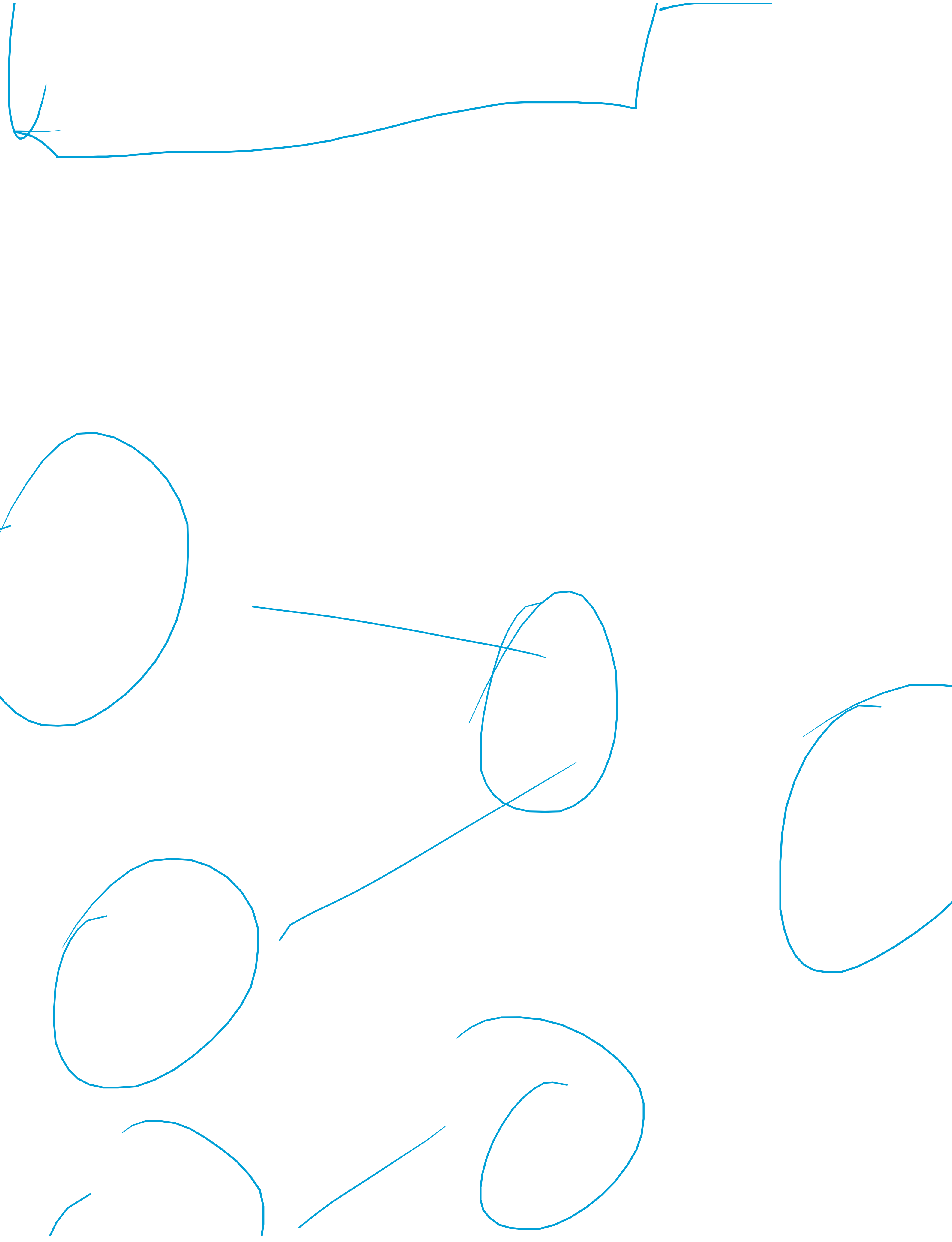



32-63
0-31




3)


X1
X2
X3




tenso

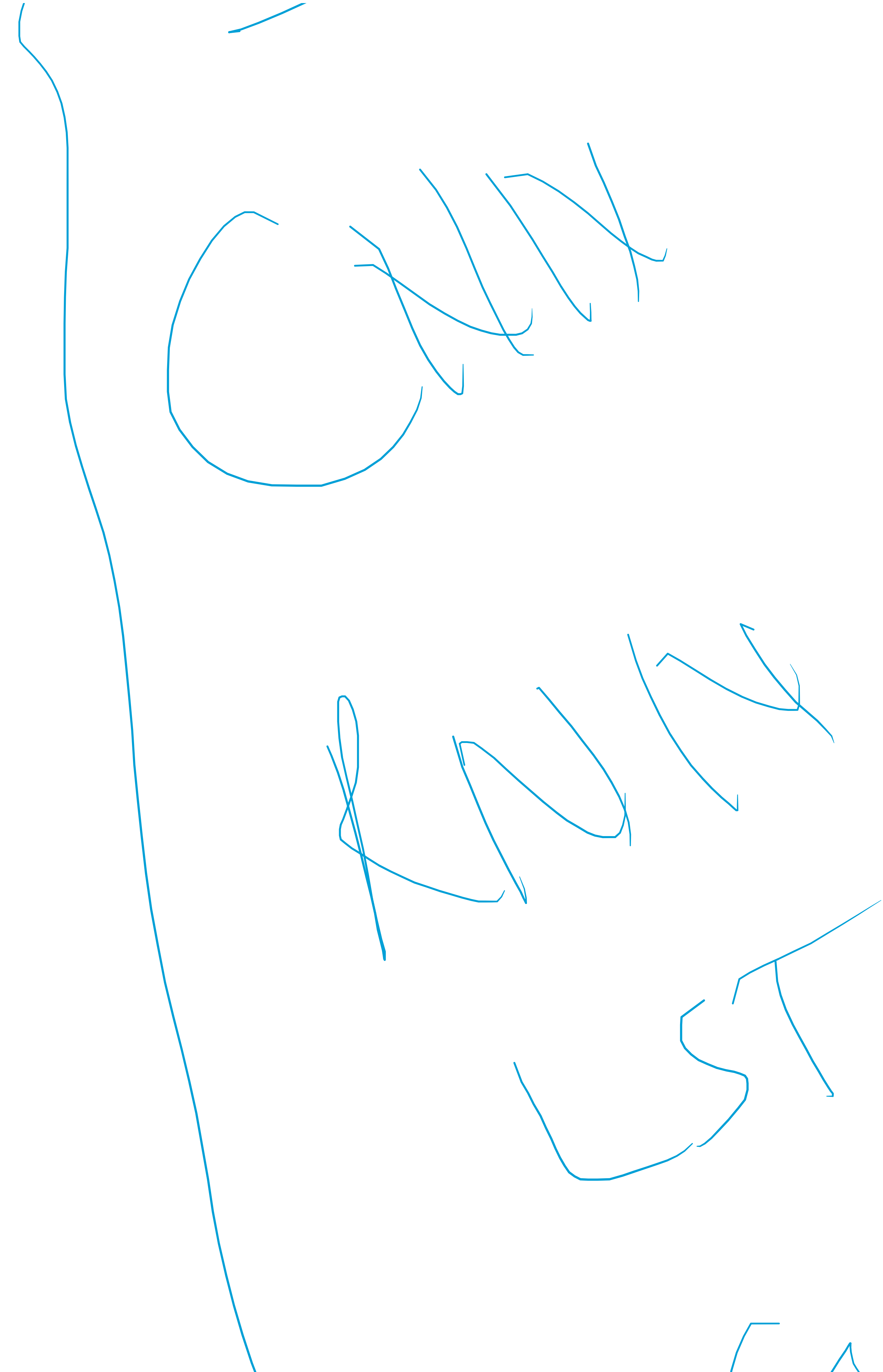

flow

wrapped

Ksa

das

lin



Kilces

t f

(−8,

Vali